

# PRISM-Loc: Three LiDAR Localization Backends Behind One ROS 2 Contract, with Middleware-Free Estimator Cores

Jungmo Kang<sup>1</sup>

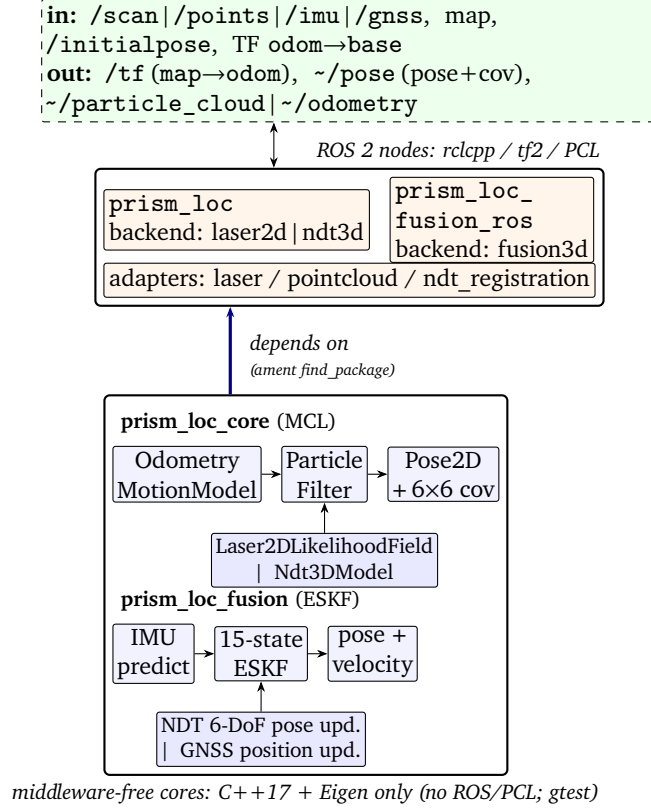
<sup>1</sup>Independent Researcher, Seoul, Republic of Korea

Widely used open-source tools for map-based LiDAR localization of mobile robots are still largely organized by sensor suite: a 2D scanner typically calls for an AMCL-style particle filter over an occupancy grid, a 3D LiDAR for an NDT-based tracker over a point-cloud map, and an inertial- or GNSS-aided platform for a Kalman fusion node, often with different topics, parameters, initialization, and recovery behavior. Existing libraries already share one particle-filter core across 2D and 3D sensor models, and other systems already relocalize a robot by branch-and-bound scan matching. We describe *PRISM-Loc*, an open-source ROS 2 localization stack that places three established estimator families behind one node contract: `laser2d`, a likelihood-field Monte Carlo localizer; `ndt3d`, a planar NDT-MCL backend (Saarinen et al., 2013) that reuses the same particle filter with a Gaussian-voxel measurement model; and `fusion3d`, a 15-state error-state Kalman filter fusing LiDAR, IMU, and RTK-GNSS. Every backend publishes the same map→odom transform and pose with covariance and accepts the same `/initialpose` seeding, all filter mathematics live in Eigen-only C++17 cores that are tested without a ROS runtime, and `laser2d` relocalizes a kidnapped robot on request (at startup or by a service call) by deterministic branch-and-bound scan matching over a map-sized search window restricted to cells not marked occupied, keeping the top- $K$  candidate poses and committing one only after several odometry-chained scans concentrate a normalized heuristic evidence score on it, and otherwise reporting the place as ambiguous. We claim no new estimator theory and no new verification principle; the contribution is this particular integration, released as software. In seeded closed-loop experiments on *synthetic* data that drive the actual cores, the 2D tracker holds a steady-state mean position error of 0.105 m, branch-and-bound relocalization succeeds on 97 of 100 kidnapped-robot queries in a 6 m room with zero false accepts at its default threshold, on 40 m and 80 m maps with repeated structure the single-scan rule accepts a wrong pose on 6 and 15 of 120 queries, which multi-scan verification reduces to 0 and 1 of 120 (pooled 95% Wilson interval 0.1–2.3%) while committing as many correct poses (242 against 241 of 300), and the ESKF bounds a 10 s aiding dropout to 0.125 m RMSE, although a 100-run NEES test shows that its covariance is overconfident (full-state NEES above the 95% interval at 89 of 120 epochs, mainly in attitude; bias blocks consistent). No external baseline and no real or public data are used, and no field-accuracy claim is made. Code: [https://github.com/kjungmo/prism\\_loc](https://github.com/kjungmo/prism_loc).

*Keywords:* mobile robots, LiDAR localization, Monte Carlo localization, NDT, error-state Kalman filter, ROS 2

## 1. Introduction

Localizing a mobile robot against a prior map is, on paper, a solved problem, but the widely used open-source tools are still largely organized by sensor suite. A team fielding a 2D LiDAR typically



**Figure 1: PRISM-Loc overview.** Two middleware-free estimator cores (bottom, C++17 + Eigen only): `prism_loc_core` runs the MCL loop (ODOMETRYMOTIONMODEL → PARTICLEFILTER, corrected by either the LASER2DLIKELIHOODFIELD or the NDT3DMODEL measurement model, plus a branch-and-bound relocalizer), and `prism_loc_fusion` runs a 15-state ESKF (IMU prediction, corrected by NDT 6-DoF pose and GNSS position updates). The ROS 2 node packages (top) consume the cores upward through `ament`, select a backend at launch, bridge messages through adapters, and expose one shared contract: a map→odom TF, a pose-with-covariance output, and an /initialpose seed input.

reaches for `nav2_amcl`, a particle filter that tracks a planar pose against an occupancy grid and nothing else. A team with a 3D LiDAR may reach instead for an `hdl_localization`-style tracker built around 3D NDT registration (Koide et al., 2019). A team that must fuse inertial and satellite measurements may reach for a `robot_localization`-style node (Moore and Stouch, 2016), which composes an EKF over odometry, IMU, and GNSS but leaves the map-relative LiDAR measurement to another node. These tools differ in topic interface, parameter vocabulary, initialization, and recovery behavior, so a practitioner who moves from a 2D platform to a 3D one, or who adds a GNSS receiver, must re-integrate much of the localization layer.

The split is not universal, and several efforts reduce it. The survey of the ROS 2 navigation maintainers (Macenski et al., 2023) catalogs the shortcomings of AMCL and reports a proposed design for a localization framework that builds 2D and 3D localization systems from modular components. Beluga (Ekumen, Inc., 2022; Puga et al., 2024) is a ROS-agnostic C++17 Monte Carlo localization library whose ROS 2 nodes offer interface parity with `nav2_amcl` and pair one particle-filter core with likelihood-field and 2D and 3D NDT sensor models; Autoware’s pose-estimator arbiter (Autoware Foundation, 2024) launches several heterogeneous pose estimators and provides a mechanism to stop and resume them; and fuse (Locus Robotics, 2018) and MOLA (Blanco-Claraco, 2025) provide modular estimation frameworks. How a stack initializes, degrades, and recovers also matters in deployment: the Nav2 marathon deployment (Macenski et al., 2020) names low localization confidence as one of

the two causes behind most of its recovery triggers. Classical MCL recovers from such failures through stochastic mechanisms, random particle injection or dual sampling (Fox et al., 1999; Thrun et al., 2001), which are hard to reproduce in a test; later systems add failure detection (Akai, 2023), keep several particle populations seeded by map matching (García et al., 2023), or reset a tracker from a branch-and-bound global search (Koide, 2021; Sasaki, 2026).

We describe *PRISM-Loc*, an engineering integration that packages three map-based LiDAR localization backends behind one ROS 2 node contract (Figure 1): *laser2d*, a 2D likelihood-field Monte Carlo localizer against an occupancy grid; *ndt3d*, a 3D NDT-MCL backend (Saarinen et al., 2013) that reuses the same particle filter with a voxel-Gaussian measurement model over a prior point cloud, sampling the planar  $(x, y, \theta)$  pose; and *fusion3d*, a 15-state error-state Kalman filter (ESKF) that fuses 3D LiDAR, IMU, and RTK-GNSS with IMU-bias estimation. All three present the same interface, a `map`→`odom` transform, a pose with  $6 \times 6$  covariance on a private `~/pose` topic that differs between the two nodes only by namespace and can be remapped (Table 2), and `/initialpose` seeding, so choosing between a 2D scan, a 3D cloud, and an inertial/GNSS-aided configuration is a launch-time choice of parameter or launch file rather than a change of interface. We claim no new estimator theory: likelihood-field MCL, NDT-MCL, ESKF fusion, and branch-and-bound scan matching are all established, and so are the broader patterns we use, namely one particle-filter core shared by 2D and 3D measurement models (Ekumen, Inc., 2022; Puga et al., 2024), branch-and-bound relocalization against a prior map (Hess et al., 2016; Koide, 2021), and tracking several global-localization hypotheses over time until the ambiguity is resolved (Gao et al., 2019; García et al., 2023). We are not aware of a released package that combines exactly the three backends below behind one contract with a relocalizer that can decline to commit; that combination is the contribution, delivered as usable open-source software. Table 1 lists the closest prior work for each component.

- *One Pluggable Contract*: three estimator families (2D likelihood-field MCL, 3D NDT-MCL, and LiDAR+IMU+RTK-GNSS ESKF fusion) present one ROS 2 `map`→`odom` contract, so the sensor suite selects a backend, not a stack with a different interface. Beluga already serves its likelihood-field and NDT particle filters through one node interface (Ekumen, Inc., 2022; Puga et al., 2024); the addition here is an inertial/GNSS Kalman backend that shares the same `map`→`odom` TF edge, pose message type, and `/initialpose` seeding (*laser2d* and *ndt3d* are chosen by one backend parameter, *fusion3d* by its own launch file); its covariance is overconfident in our test, so we release it as experimental (Section 5). The release adds fail-fast configuration checks, a startup watchdog that names the input that has gone silent, and a single-source parameter reference.
- *Middleware-Free, Deterministic Cores*: all filter mathematics sit behind an Eigen-only boundary with no `rclcpp`, `tf2`, or `PCL`, and all sampling routes through one seeded generator, so every algorithmic claim is exercised by 63 deterministic unit and integration tests and by closed-loop synthetic experiments that compile the cores directly. Middleware-independent estimator cores are established practice (Pedrosa, 2019; Puga et al., 2024); we list this as a property of the artifact, not as a research contribution.
- *Global Relocalization That Can Abstain*: for *laser2d*, the branch-and-bound correlative scan matcher of Hess et al. (2016), run over a max-pooled likelihood pyramid, finds candidate poses for a kidnapped robot from one scan and re-seeds the MCL particle set once a candidate is verified, in place of stochastic random-injection recovery. By the standard branch-and-bound argument of Hess et al. (2016), restated for our search set in Section 3.4, the search returns the best correlative score over the non-occupied (free or unknown) cells of the rotation–translation lattice; this is the discrete optimum of a surrogate score, not an optimal pose estimate. The search window is sized from the map, so the search is global up to a configurable compute cap

(100 m per axis by default). Because the optimum of one scan can be a look-alike place, the node keeps the top- $K$  distinct candidates, propagates each by odometry and re-scores it by a local branch-and-bound match, and commits one only when it holds most of a normalized heuristic evidence score over several scans, reporting the place as ambiguous otherwise. Tracking several hypotheses (Gao et al., 2019; García et al., 2023) and requiring evidence to persist over several scans and some motion before declaring a robot localized (Adurthi, 2023) are established, and concurrent work makes an abstaining decision explicit for LiDAR-to-BIM initialization (Zhang et al., 2026b); what is specific here is this combination on 2D occupancy grids, wired to re-seed the shared MCL core.

We validate PRISM-Loc at the level of correctness and closed-loop behavior, not field accuracy. On seeded synthetic worlds generated to match the filters’ assumptions, the `laser2d` tracker holds a steady-state mean position error of 0.105 m while KLD-adaptive resampling shrinks the particle set from the 2000-particle ceiling to the 500-particle floor; branch-and-bound relocalization recovers 97 of 100 kidnapped-robot queries in a 6 m room under up to 80% beam clutter with zero false accepts at its default acceptance threshold, and on 40 m and 80 m maps recovers 50 and 41 of 60 poses that lie outside the fixed window of our first release, which recovered 2 and 0; there, however, a single scan accepts a wrong pose on 6 and 15 of 120 queries because repeated shelves look alike, and multi-scan verification brings this to 0 and 1 of 120 without losing correct fixes overall; and the ESKF limits a 10 s loss of all aiding to 0.125 m position RMSE, while a 100-run Monte Carlo NEES test shows that its covariance is overconfident, mainly in attitude. These numbers demonstrate that the assembled pipelines behave as intended; they are not a benchmark. We run no external baseline and use no real or public data, so we make no claim of accuracy or robustness relative to `nav2_amcl`, Beluga, MH-AMCL, or the global-localization methods of Section 2; that comparison is planned (Section 5).

## 2. Related Work

**Monte Carlo Localization.** Particle-filter localization was introduced by Dellaert et al. (1999) and Fox et al. (1999), who represent the pose posterior as a weighted sample set updated by motion-model prediction, sensor-likelihood weighting, and resampling, and who demonstrated global localization from a uniform prior on real robots. Thrun et al. (2001) later hardened MCL against accurate sensors and the kidnapped-robot problem via Mixture-MCL’s dual sampler, and Fox (2003) made the sample size adaptive through KLD-sampling; the odometry motion model and likelihood-field sensor model we use are the textbook forms of Thrun et al. (2005). The particle-filter core of PRISM-Loc is a direct descendant of this line and of its `nav2_amcl` lineage; we contribute no new filter theory. Sharing one such core across measurement models is not new either. Beluga (Ekumen, Inc., 2022; Puga et al., 2024) is a ROS-agnostic, extensible C++17 MCL library; its ROS 2 nodes offer interface parity with `nav2_amcl` and combine one particle-filter core with likelihood-field, beam, and 2D and 3D NDT sensor models, and its 3D NDT node estimates a full  $SE(3)$  pose, which is more general than the planar state of our `ndt3d` backend. Earlier, `mc1_3d1` (Watanabe, 2016) implemented 3D/6-DoF point-cloud MCL against a reference cloud for ROS, and Martín Rico et al. (2023) replace Nav2’s AMCL with an MCL over Gridmaps and Octomaps for non-planar terrain. Other work makes MCL recovery more reliable: Akai (2023) estimates the reliability of the MCL estimate to detect failure and integrates a global localization method through importance sampling for quick re-localization, and MH-AMCL (García et al., 2023) maintains several particle populations, created by multi-scale map matching, and outputs the most reliable one. PRISM-Loc instead delegates recovery to a deterministic branch-and-bound search (Section 3.4) followed by multi-scan verification (Section 3.5).

**NDT Representations and Measurement Models.** Biber and Straßer (2003) introduced the Normal

Distributions Transform, modeling a scan as per-cell Gaussians and reducing registration to Newton optimization of a smooth score; [Magnusson et al. \(2007\)](#) generalized the representation to 3D voxel grids for field deployment on mining vehicles. Both use NDT for deterministic scan-to-scan or scan-to-map registration that requires a reasonable initial guess. [Saarinen et al. \(2013\)](#) brought the NDT into Monte Carlo localization (NDT-MCL), using it as the representation of both the map and the sensor data. The `ndt3d` backend of PRISM-Loc follows this NDT-MCL idea in a simplified form: it scores raw subsampled points against the Gaussian-voxel map and reads the score out as a per-particle likelihood inside the shared MCL core, trading the quadratic convergence of Newton alignment for the multi-hypothesis robustness of the particle filter, so the 2D and 3D backends differ only in their measurement model.

**LiDAR-Inertial Fusion and Robot Software Systems.** [Solà \(2017\)](#) is the mathematical basis of the `fusion3d` backend: the nominal/error-state decomposition, local angular-error parameterization, and predict-correct-inject-reset cycle are implemented essentially as derived there. On the systems side, [Moore and Stouch \(2016\)](#) established the configurable EKF/UKF fusion node for ROS, [Wan et al. \(2018\)](#) demonstrated an industrial ESKF fusing GNSS-RTK, LiDAR, and IMU at centimeter accuracy as a production system tied to its own pre-built LiDAR maps, and Nav2 ([Macenski et al., 2020](#)) treats localization as an external, swappable component. Tightly-coupled LiDAR-inertial odometry such as FAST-LIO ([Xu and Zhang, 2021](#)), and applied 3D map-based tracking systems ([Koide et al., 2019](#)), are complementary: they produce ego-motion and maps, whereas PRISM-Loc localizes within a prior map. Several systems already place more than one estimator behind a common interface. `fuse` ([Locus Robotics, 2018](#)) is a ROS framework for sensor fusion by nonlinear least squares with plugin-based sensor and motion models; Autoware’s pose-estimator arbiter ([Autoware Foundation, 2024](#)) launches NDT scan matching, YabLoc, Eagleye, and a landmark-based localizer, selected by a `pose_source` argument, and provides a stop/resume mechanism driven by pluggable switching rules (the shipped rule enables all of them); and MOLA ([Blanco-Claraco, 2025](#)) is a framework for LiDAR odometry, map manipulation, and localization whose pipelines are assembled from reusable blocks. Estimator cores kept free of the middleware are likewise established practice ([Pedrosa, 2019](#); [Puga et al., 2024](#)). PRISM-Loc is one narrow instance of this pattern: it places exactly three estimator families behind one ROS 2 topic and TF contract, each implemented as a library that is unit-testable without a middleware runtime. It covers a small part of the direction set out by the modular 2D/3D localization framework whose design [Macenski et al. \(2023\)](#) report as proposed, and it is not a general localization framework.

**Global (Kidnapped-Robot) Localization.** [Olson \(2009\)](#) formulated 2D scan matching as exhaustive correlative search over a rasterized log-likelihood table with coarse-to-fine multi-resolution pruning, and [Hess et al. \(2016\)](#) bounded that search with branch-and-bound over a precomputed grid pyramid to run loop closure in real time inside Cartographer. PRISM-Loc borrows exactly this branch-and-bound-over-pyramid primitive and applies it to relocalization against a known static map, a setting that existing tools already serve. `hdl_global_localization` ([Koide, 2021](#)) offers a 2D grid-map branch-and-bound engine, among others, as a ROS service, which `hdl_localization` ([Koide et al., 2019](#)) calls to reset the pose of its UKF-based tracker, whose corrections come from NDT scan matching; the global-localization tooling of `lidar_localization_ros2` ([Sasaki, 2026](#)), documented in 2026, runs map-wide branch-and-bound over an occupancy grid, returns ranked candidates, and re-seeds an NDT or GICP tracker through `/initialpose` behind safety guards that include consecutive confirmation samples and a cross-check against along-corridor aliases. 3D-BBS ([Aoki et al., 2024](#)) extends branch-and-bound global localization to 3D point-cloud maps. [Adurthi \(2023\)](#) uses a hierarchical, max-pooled scan-to-map matcher to enrich the particles of a 3D-LiDAR particle filter before resampling. Other single-scan 2D methods include CBGL ([Filotheou, 2024](#)) and the method of [Zhang et al. \(2026a\)](#), which samples positional hypotheses over traversable space

with a rapidly-exploring random tree, a restriction similar in spirit to our rule that candidates lie in non-occupied cells. Feeding global-matching or global-localization results into a particle filter is itself established (Adurthi, 2023; Akai, 2023; García et al., 2023); what PRISM-Loc adds is a specific wiring to its own MCL core: a 2D branch-and-bound search over the non-occupied cells of a map-sized window, verified over several scans (Section 3.5) before it re-seeds the particle set once, in place of the heuristic recovery of classical MCL (Fox et al., 1999; Thrun et al., 2001).

**Multi-Hypothesis Verification and Abstention.** Deferring commitment until later observations disambiguate a place is also established. Gao et al. (2019) extract candidate poses from successive 2D laser scans and resolve them with an extended multiple-hypothesis tracker whose likelihood combines consecutive candidate poses with odometry; MH-AMCL (García et al., 2023) keeps several particle populations alive and selects the most reliable; Adurthi (2023) declares a vehicle localized only after the particle spread stays small over consecutive scans and the vehicle has moved; Reliable-loc (Zou et al., 2024) monitors sequential pose uncertainty to switch localization modes in large street scenes; and Akai (2023) estimates the reliability of an MCL estimate to detect failure. Concurrently with this work, Zhang et al. (2026b) proposed a selective layer for LiDAR-to-BIM global initialization that aggregates candidates from several submaps and issues typed COMMIT, DEFER, or AMBIGUOUS decisions, which symmetry-breaking motion can upgrade to COMMIT. The verifier of Section 3.5 is a specific and simpler instance of this idea for 2D occupancy grids: top- $K$  first-scan branch-and-bound modes, propagated by odometry and re-scored by a local branch-and-bound match, with an explicit ambiguous outcome before the MCL core is re-seeded. Its evidence score is a tuned heuristic, not a calibrated probability.

### 3. PRISM-Loc: One Localization Contract over Three Middleware-Free Estimators

PRISM-Loc separates *what* a localization node promises its consumers from *how* a pose is estimated. We first state the estimation problem and the output contract shared by all backends, then describe the two-layer architecture that realizes the contract (Section 3.1), the shared particle-filter core (Section 3.2), its two measurement models (Section 3.3), the branch-and-bound relocalizer (Section 3.4), the error-state fusion filter (Section 3.6), and the initialization and diagnostic layer (Section 3.7). Every equation below is transcribed from the C++ sources listed in Section C.

**Problem Setup: One Contract, Interchangeable Estimators.** Let  $m$  be a prior map (an occupancy grid or a point cloud),  $\mathbf{u}_{1:t}$  the odometry or inertial inputs, and  $\mathbf{z}_{1:t}$  the exteroceptive measurements (2D scans, 3D clouds, GNSS fixes). Each backend  $b \in \mathcal{B} = \{\text{laser2d}, \text{ndt3d}, \text{fusion3d}\}$  approximates the Bayes-filter belief over the robot state  $\mathbf{x}_t$  (Thrun et al., 2005),

$$\text{bel}_b(\mathbf{x}_t) = \eta p_b(\mathbf{z}_t | \mathbf{x}_t, m) \int p_b(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1}) \text{bel}_b(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1}, \quad (1)$$

with a particle set for the two MCL backends and a Gaussian error state for fusion3d. Whatever the representation, a backend reports a point estimate  $\hat{\mathbf{T}}_{\text{map},\text{base}}$  with a  $6 \times 6$  covariance  $\hat{\Sigma}_t$  and, so as not to fight the odometry source that owns `odom→base_link`, broadcasts only the correction transform

$$\hat{\mathbf{T}}_{\text{map},\text{odom}} = \hat{\mathbf{T}}_{\text{map},\text{base}} \oplus \mathbf{T}_{\text{odom},\text{base}}^{-1}, \quad (2)$$

where  $\oplus$  is pose composition and  $\mathbf{T}_{\text{odom},\text{base}}$  is read from TF at the measurement stamp. Equation (2) guarantees the round trip  $\hat{\mathbf{T}}_{\text{map},\text{odom}} \oplus \mathbf{T}_{\text{odom},\text{base}} = \hat{\mathbf{T}}_{\text{map},\text{base}}$ , which ROS-free unit tests assert to  $10^{-9}$  on  $SE(2)$  and  $SE(3)$ . The design objective is that the tuple (TF edge, pose message on the private `~/pose` topic, seeding topic) is the same for all  $b \in \mathcal{B}$ , up to the node namespace under which `~/pose` resolves (Table 2), so that the sensor suite, not the software stack, selects  $b$ .

**Table 1: Closest prior work for each component of PRISM-Loc.** The last column states what PRISM-Loc does, not what the cited systems lack; a component is claimed as a difference only where the cited sources document the alternative. “Concurrent” marks a preprint posted in July 2026.

| Component                                                       | Closest prior work                                                                                                                                                                        | In PRISM-Loc                                                                                                                                                                                                                          |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| One particle-filter core for 2D likelihood-field and NDT models | Beluga (Puga et al., 2024); NDT-MCL (Saarinen et al., 2013)                                                                                                                               | No methodological difference. ndt3d samples a planar pose (Beluga’s 3D NDT node estimates $SE(3)$ (Ekumen, Inc., 2022)) and scores raw points against the NDT map.                                                                    |
| Several estimators behind one interface                         | fuse (Locus Robotics, 2018); Autoware pose-estimator arbiter (Autoware Foundation, 2024); MOLA (Blanco-Claraco, 2025)                                                                     | Exactly three backends, 2D MCL, planar NDT-MCL, and a LiDAR+IMU+RTK-GNSS ESKF, behind one map→odom contract chosen at launch.                                                                                                         |
| Middleware-free estimator core                                  | Beluga (Puga et al., 2024); iris_lama (Pedrosa, 2019)                                                                                                                                     | No methodological difference; one seeded generator makes the tests and experiments bit-reproducible.                                                                                                                                  |
| Branch-and-bound relocalization against a prior map             | Hess et al. (2016); hdl_global_localization (Koide, 2021); lidar_localization_ros2 (Sasaki, 2026); 3D-BBS (Aoki et al., 2024); Adurthi (2023); MH-AMCL (García et al., 2023); Akai (2023) | Lattice-exact 2D branch-and-bound over the non-occupied cells of a map-sized window, verified over several scans before a single re-seed of the shared MCL core; feeding global matches into a particle filter is itself established. |
| Hypotheses verified before commitment                           | Gao et al. (2019); MH-AMCL (García et al., 2023); Adurthi (2023); concurrent: Zhang et al. (2026b)                                                                                        | First-scan top-K branch-and-bound modes, odometric propagation, local re-scoring, and an explicit ambiguous outcome before re-seeding; the evidence score is heuristic.                                                               |
| Error-state Kalman fusion of LiDAR, IMU, and GNSS               | Solà (2017); Wan et al. (2018)                                                                                                                                                            | No methodological difference; the filter is exposed under the same contract as the MCL backends, and its covariance is overconfident in our test (Table 7).                                                                           |

### 3.1. Separating Estimators from Middleware: Two Layers, One Contract

PRISM-Loc is organized as two layers realized by four packages (Figure 1). The lower layer holds the estimator cores: `prism_loc_core` implements the MCL family and the relocalizer, and `prism_loc_fusion` implements the ESKF. Both are plain C++17 libraries whose only third-party dependency is Eigen; each `CMakeLists.txt` links `Eigen3::Eigen` alone, with no reference to `rclcpp`, `tf2`, or `PCL`. The upper layer holds the ROS 2 node packages `prism_loc` (the `laser2d` and `ndt3d` backends, selected by a backend string parameter) and `prism_loc_fusion_ros` (the `fusion3d` backend). Only these depend on the middleware, and the dependency direction is strictly upward.

The split buys three properties. First, *unit-testability without a running graph*: each core builds and tests with a plain toolchain and GoogleTest, and continuous integration exercises this ROS-free path directly. Second, *determinism*: all sampling routes through a single seeded `std::mt19937_64` wrapper (ROS parameter `seed`, default 42), so a prediction, weighting, or resampling step is reproducible bit for bit, which makes the core tests assertable rather than merely statistical. Third, *one output contract* (Table 2): every backend broadcasts map→odom per Equation (2), publishes a `PoseWithCovarianceStamped` on `~/pose`, and accepts `/initialpose`; the MCL nodes add a particle cloud and the fusion node

**Table 2: Per-backend topic contract.** All backends additionally subscribe `/initialpose` and the TF `odom→base_link`, and broadcast `map→odom` by Equation (2) with the stamp future-dated by `transform_tolerance` (default 0.1s). The private topic `~/pose` resolves under the node name (`/prism_loc/pose` for `laser2d` and `ndt3d`, `/prism_loc_fusion/pose` for `fusion3d`) and can be remapped.

| Backend               | Estimator core                             | Inputs                                                                                                                  | Outputs                                                |
|-----------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>laser2d</code>  | MCL + likelihood field<br>(Equation (7))   | <code>/scan</code> , <code>/map</code><br>( <code>OccupancyGrid</code> )                                                | <code>~/pose</code> ,<br><code>~/particle_cloud</code> |
| <code>ndt3d</code>    | MCL + NDT score<br>(Equation (8))          | <code>/points</code> , <code>.pcd</code> map file                                                                       | <code>~/pose</code> ,<br><code>~/particle_cloud</code> |
| <code>fusion3d</code> | 15-state ESKF (Equations (13)<br>and (14)) | <code>/points</code> , <code>/imu</code> , <code>/gnss</code><br>( <code>NavSatFix</code> ), <code>.pcd</code> map file | <code>~/pose</code> , <code>~/odometry</code>          |

an Odometry message. Thin adapters convert `LaserScan`, `OccupancyGrid`, `PointCloud2`, and `.pcd` files into core value types, and in `fusion3d` an adapter wraps PCL’s NDT registration to turn a live cloud into a 6-DoF pose measurement, so PCL appears only inside adapters. QoS profiles follow publisher semantics: sensor streams are subscribed with `SensorDataQoS` (best-effort), while the occupancy grid is subscribed `transient_local` and `reliable` so the single latched map from `map_server` reaches a late-joining node.

### 3.2. Sampling and Weighting Particles: One MCL Core for 2D and 3D

Both pose-tracking backends share one particle filter over the planar pose  $\mathbf{x} = (x, y, \theta)$  (Pose2D), with belief  $\{(\mathbf{x}_t^{[i]}, w_t^{[i]})\}_{i=1}^{N_t}$ . The recursion is the one wired in `localization_node.cpp` and summarized in Algorithm 1.

**Motion Model.** `OdometryMotionModel` implements the sampling form of the odometry model (Thrun et al., 2005). Consecutive odometry poses are decomposed into  $(\delta_{r1}, \delta_t, \delta_{r2})$ , with  $\delta_{r1} = 0$  whenever  $\delta_t < 10^{-3}$  m (pure-rotation guard), and each particle is advanced by the noise-corrupted increment

$$\begin{aligned}\hat{\delta}_{r1} &= \delta_{r1} - \varepsilon(\alpha_1 \delta_{r1}^2 + \alpha_2 \delta_t^2), & \hat{\delta}_{r2} &= \delta_{r2} - \varepsilon(\alpha_1 \delta_{r2}^2 + \alpha_2 \delta_t^2), \\ \hat{\delta}_t &= \delta_t - \varepsilon(\alpha_3 \delta_t^2 + \alpha_4 (\delta_{r1}^2 + \delta_{r2}^2)),\end{aligned}\tag{3}$$

where  $\varepsilon(\sigma^2)$  draws zero-mean Gaussian noise of variance  $\sigma^2$  and  $\alpha_{1..4}$  are the `alpha1`–`alpha4` parameters.

**Weighting.** Given a measurement model with log-likelihood  $\ell(\mathbf{x})$  (Section 3.3), the correct step of `ParticleFilter` replaces each weight by

$$w^{[i]} = \frac{\exp(\ell(\mathbf{x}^{[i]}) - \ell_{\max})}{\sum_j \exp(\ell(\mathbf{x}^{[j]}) - \ell_{\max})}, \quad \ell_{\max} = \max_j \ell(\mathbf{x}^{[j]}),\tag{4}$$

subtracting the per-step maximum for numerical stability. The weights are overwritten rather than multiplied into the previous ones; this coincides with the standard recursion after every resampling step (which leaves uniform weights), and otherwise discards the previous step’s weights, a simplification we note rather than hide.

**KLD-Adaptive Resampling.** Resampling is triggered only when the effective sample size  $N_{\text{eff}} = (\sum_i w^{[i]})^2 / \sum_i (w^{[i]})^2$  falls below  $\tau N$  (`resample_threshold`  $\tau = 0.5$ ). Samples are then drawn with probability proportional to weight and dropped into a histogram with bins of `kld_bin_xy` in  $(x, y)$

**Algorithm 1:** One laser2d/ndt3d filter update, as wired in `localization_node.cpp`


---

**Input:** particle set  $\mathcal{X}$ ; previous and current odometry poses  $\mathbf{o}_{t-1}, \mathbf{o}_t$ ; measurement model  $\ell$ ; gates  $d_{\min}, a_{\min}$

```

1 if  $\|\mathbf{o}_t - \mathbf{o}_{t-1}\|_{xy} \geq d_{\min}$  or  $|\theta(\mathbf{o}_t) - \theta(\mathbf{o}_{t-1})| \geq a_{\min}$  or a re-seed forced an update then
2   for  $i \leftarrow 1$  to  $|\mathcal{X}|$  do
3      $\mathbf{x}^{[i]} \leftarrow$  sample Equation (3) from  $(\mathbf{o}_{t-1}, \mathbf{o}_t)$  applied to  $\mathbf{x}^{[i]}$ ;
4     compute weights  $w^{[i]}$  by Equation (4);
5     if  $N_{\text{eff}} < \tau|\mathcal{X}|$  then
6        $\mathcal{X} \leftarrow$  KLD-resample until  $n \geq \max(M_k, N_{\min})$  (Equation (5)) or  $n = N_{\max}$ ;
7      $\mathbf{o}_{t-1} \leftarrow \mathbf{o}_t$ ;

```

**Output:**  $(\hat{x}, \hat{y}, \hat{\theta})$  and covariance by Equation (6);  $\hat{\mathbf{T}}_{\text{map,odom}}$  by Equation (2)

---

and `kld_bin_yaw` in  $\theta$ . Following Fox (2003), after  $k > 1$  distinct bins have been occupied the target sample count is

$$M_k = \frac{k-1}{2\epsilon_{\text{kld}}} \left( 1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right)^3, \quad (5)$$

and drawing stops once  $n \geq \max(M_k, N_{\min})$  or  $n = N_{\max}$ , with  $\epsilon_{\text{kld}}$  (`kld_err`),  $z_{1-\delta}$  (`kld_z`), and  $[N_{\min}, N_{\max}]$  (`min_particles`, `max_particles`) as parameters. The resampled set receives uniform weights.

**Pose Estimate.** The reported pose is the weighted mean of  $(x, y)$  with the circular mean of  $\theta$ ,

$$\hat{x} = \sum_i w^{[i]} x^{[i]}, \quad \hat{y} = \sum_i w^{[i]} y^{[i]}, \quad \hat{\theta} = \text{atan2}\left(\sum_i w^{[i]} \sin \theta^{[i]}, \sum_i w^{[i]} \cos \theta^{[i]}\right), \quad (6)$$

and the  $6 \times 6$  covariance carries the weighted planar second moments in  $(x, y)$  and  $\theta$ , while  $z$ , roll, and pitch are marked unobserved with variance  $10^6$ .

### 3.3. Scoring Observations: Likelihood-Field and NDT Measurement Models

A backend differs from the other only through a `MeasurementModel`, whose single virtual method returns the unnormalized log-likelihood  $\ell(\mathbf{x})$  of the current observation given a candidate base pose.

**2D Likelihood Field.** `Laser2DLikelihoodField` is the likelihood-field model (Thrun et al., 2005). At construction the occupancy grid (cells with value  $\geq 50$  occupied) is converted into a field  $d(\cdot)$  holding the Euclidean distance to the nearest occupied cell, computed exactly by the separable squared-distance transform of Felzenszwalb and Huttenlocher (2012) and clamped at `likelihood_max_dist`. For a scan with ranges  $r_j$  at bearings  $\phi_j$ , the model visits every  $s$ -th beam,  $s = \max(1, \lfloor n/\text{max\_beams} \rfloor)$ , discards ranges outside  $(r_{\min}, r_{\max})$ , and scores

$$\ell_{2D}(\mathbf{x}) = \sum_{j \in \mathcal{J}} \log \max\left(z_{\text{hit}} \exp\left(-d(\mathbf{e}_j(\mathbf{x}))^2 / 2\sigma_{\text{hit}}^2\right) + z_{\text{rand}}/r_{\max}, 10^{-12}\right), \quad (7)$$

where  $\mathbf{e}_j(\mathbf{x})$  is the beam endpoint  $r_j(\cos \phi_j, \sin \phi_j)$  transformed into the map through  $\mathbf{x}$  composed with the sensor mounting pose.

**3D NDT Score.** `Ndt3DModel` scores a 3D point cloud against a Normal Distributions Transform map (Biber and Straßer, 2003; Magnusson et al., 2007). `NdtMap` voxelizes the prior cloud at resolution `ndt_resolution`; each voxel  $v$  with at least `voxel_min_points` points stores its mean  $\mu_v$  and the

inverse of its covariance after inflating every eigenvalue to at least  $10^{-3}$  of the largest. With  $\mathbf{T}(\mathbf{x})$  the 3D lift of the planar particle at the fixed height `base_height` composed with the sensor pose, the per-particle score over a subsampled cloud  $\mathcal{P}$  (at most `max_points` points) is

$$\ell_{3D}(\mathbf{x}) = \sum_{\mathbf{p} \in \mathcal{P}} \log(s(\mathbf{T}(\mathbf{x})\mathbf{p}) + s_{\text{floor}}), \quad s(\mathbf{q}) = \exp\left(-\frac{1}{2}(\mathbf{q} - \boldsymbol{\mu}_{v(\mathbf{q})})^\top \boldsymbol{\Sigma}_{v(\mathbf{q})}^{-1}(\mathbf{q} - \boldsymbol{\mu}_{v(\mathbf{q})})\right), \quad (8)$$

with  $s(\mathbf{q}) = 0$  when  $\mathbf{q}$  falls in an empty voxel and  $s_{\text{floor}} = 10^{-6}$ . This is a simplified form of NDT-MCL (Saarinen et al., 2013), which represents both the map and the scan by NDTs; here raw points are scored against the NDT map, a score of the kind that Newton registration optimizes is read out as a particle likelihood, and the 3D backend estimates the same  $(x, y, \theta)$  as the 2D one.

### 3.4. Relocalizing by Branch-and-Bound: Deterministic Kidnapped-Robot Recovery

Recovery from a lost or kidnapped estimate is handled by `BranchAndBoundMatcher`, a correlative scan matcher (Olson, 2009) bounded by a max-pooled pyramid as in Hess et al. (2016). At construction the likelihood field (clamped at  $4\sigma_{\text{hit}}$ ) is converted into a probability grid and a pyramid of  $D + 1$  levels (`bbs_max_depth D`),

$$P_0(c) = \exp(-d(c)^2/2\sigma_{\text{hit}}^2), \quad P_h(c_x, c_y) = \max_{0 \leq a, b < 2^h} P_0(c_x + a, c_y + b), \quad (9)$$

where cells outside the grid count as zero;  $P_h$  is built by doubling,  $P_h(c) = \max\{P_{h-1}(c), P_{h-1}(c + 2^{h-1}\mathbf{e}_x), P_{h-1}(c + 2^{h-1}\mathbf{e}_y), P_{h-1}(c + 2^{h-1}\mathbf{1})\}$ . A search at center  $\mathbf{c}$  with per-axis half-windows  $(L_x, L_y)$  in cells enumerates rotations  $\theta_k \in \Theta$  over  $\pm \text{bbs\_angular\_window}$  at `bbs\_angular\_step` and the *symmetric* set of integer cell offsets  $O = \{-L_x, \dots, L_x\} \times \{-L_y, \dots, L_y\}$ . In the node's default *map-sized* mode (`bbs_global_window`), for a  $w \times h$  grid  $\mathbf{c}$  is the center of cell  $(\lfloor w/2 \rfloor, \lfloor h/2 \rfloor)$  and  $L_x = \max(1, \min(\lfloor w/2 \rfloor, \text{round}(W_{\text{max}}/\text{res})))$ ,  $L_y = \max(1, \min(\lfloor h/2 \rfloor, \text{round}(W_{\text{max}}/\text{res})))$  with the compute cap  $W_{\text{max}} = \text{bbs\_max\_linear\_window}$  (50 m). Offset  $\mathbf{o}$  then places the robot at the center of cell  $(\lfloor w/2 \rfloor, \lfloor h/2 \rfloor) + \mathbf{o}$ , so unless the cap binds, the candidate positions are exactly the centers of all grid cells (plus at most one off-grid row or column, which the feasibility rule below removes); when the cap binds, the node logs a warning that relocalization is not global. With `bbs_global_window` disabled,  $L_x = L_y = \text{round}(W/\text{res})$  about the map midpoint for the linear window  $W = \text{bbs\_linear\_window}$ . A candidate  $(k, \mathbf{o})$  is *feasible* when the robot position  $\mathbf{c} + \text{res} \cdot \mathbf{o}$  falls in an on-grid cell with occupancy below 50, i.e. a free or unknown ( $-1$ ) cell; the feasible set is  $\mathcal{F} \subseteq \Theta \times O$ . (Version 0.1 of the code anchored the roots at  $-L, -L + 2^D, \dots \leq L$  without trimming, so it enumerated the asymmetric superset  $[-L, -L + 2^D \lceil (2L + 1)/2^D \rceil]^2$ ,  $-10$  m to  $+15.5$  m about the map midpoint for the defaults, and it had no feasibility rule.) For rotation  $\theta_k$ , let  $\mathbf{g}_j^k$  be the grid cell of the  $j$ -th subsampled endpoint ( $j \in \mathcal{E}$ : every  $s$ -th beam with a valid range,  $s = \max(1, \lfloor n/\text{bbs\_max\_beams} \rfloor$ ) as in Equation (7), so  $|\mathcal{E}|$  can reach  $2 \text{bbs\_max\_beams} - 1$ ) at the center translation. A node at depth  $h$  with anchor  $\mathbf{o}$  covers the offsets  $\mathbf{o} + [0, 2^h)^2$  and is scored by

$$S_h(k, \mathbf{o}) = \sum_{j \in \mathcal{E}} P_h(\kappa_h(\mathbf{g}_j^k + \mathbf{o})), \quad (10)$$

where at  $h = 0$  out-of-grid cells contribute zero, and at  $h \geq 1$  a block that lies entirely outside the grid contributes zero while any other anchor is clamped into the grid by  $\kappa_h$ . Leaves ( $h = 0$ ) carry the exact correlative score  $S_0$ , which is the correlative surrogate of Hess et al. (2016); Olson (2009), not the MCL log-likelihood of Equation (7).

The two statements below are not new results. They restate the standard branch-and-bound argument of Hess et al. (2016), who note that the search is exact, returning the same solution as exhaustive

search, whenever the score of every inner node upper-bounds the scores of its leaves. We restate it because our search set differs from theirs in three details on which the argument depends: coarse nodes are clamped at the map border, the offset window is trimmed to be symmetric, and infeasible leaves are skipped.

**Proposition 1** (Admissible bound; cf. [Hess et al. 2016](#)). *For every node  $(k, \mathbf{o})$  at depth  $h \geq 1$  and every leaf offset  $\mathbf{o}' \in \mathbf{o} + [0, 2^h)^2$ ,  $S_h(k, \mathbf{o}) \geq S_0(k, \mathbf{o}')$ .*

*Proof.* Fix an endpoint  $j$  and let  $\mathbf{c} = \mathbf{g}_j^k + \mathbf{o}$ . If the block  $\mathbf{c} + [0, 2^h)^2$  lies entirely outside the grid, every leaf cell  $\mathbf{g}_j^k + \mathbf{o}'$  is also outside and contributes zero, so both sides are 0. Otherwise, when  $\mathbf{c}$  is inside the grid  $P_h(\mathbf{c})$  is by Equation (9) the maximum of  $P_0$  over the block, which contains the leaf cell. When a coordinate of  $\mathbf{c}$  is negative, clamping it to 0 yields a block  $[0, 2^h)$  in that coordinate that contains every in-grid cell of  $[c, c + 2^h)$ , because  $c < 0$ ; out-of-grid leaf cells contribute  $0 \leq P_h$ . Hence each endpoint's term at depth  $h$  upper-bounds its term at every leaf of the node, and summing over  $j$  gives the claim. (Without the clamp, a coarse node whose corner falls off the map would under-count and could prune the optimum; the implementation comment in `bbs.cpp` records exactly this case.)  $\square$

Algorithm 2 tiles  $\mathcal{O}$  with root anchors  $-L_x, -L_x + 2^D, \dots \leq L_x$  (and likewise in  $y$ ), expands nodes depth-first with children visited in decreasing order of their bound, discards any child whose anchor exceeds  $L_x$  or  $L_y$ , prunes a subtree whenever its bound does not exceed the best feasible leaf score found so far, and skips infeasible leaves.

**Corollary 1** (Optimality over the feasible set; cf. [Hess et al. 2016](#)). *If  $\mathcal{E} \neq \emptyset$  and  $\mathcal{F} \neq \emptyset$ , Algorithm 2 returns a candidate  $(k^*, \mathbf{o}^*) \in \mathcal{F}$  with  $S_0(k^*, \mathbf{o}^*) = \max_{(k, \mathbf{o}) \in \mathcal{F}} S_0(k, \mathbf{o})$ ; ties go to the first maximizer reached.*

*Proof.* *Coverage.* Per axis, with  $L \in \{L_x, L_y\}$ : for  $o \in \{-L, \dots, L\}$  the root anchor  $a = -L + 2^D \lfloor (o+L)/2^D \rfloor$  satisfies  $a \leq o \leq L$ , so  $o$  lies in a root block; every ancestor of the leaf  $o$  inside that block has an anchor  $\leq o \leq L$  and is never discarded by the trimming rule. Trimming therefore removes only blocks that contain no offset of  $\mathcal{O}$ , and every candidate of  $\mathcal{O}$  is a leaf of the enumerated tree; conversely every leaf has anchor  $\leq L$  in each axis and anchor  $\geq -L$  by construction, so the leaves are exactly  $\Theta \times \mathcal{O}$ . *Pruning.* The incumbent  $S^*$  is only ever set from a feasible leaf. A subtree is pruned only when its bound is  $\leq S^*$ , and by Proposition 1 (which holds for every leaf of a block, feasible or not, inside  $\mathcal{O}$  or not) no leaf below it scores more than  $S^*$ . Hence no feasible leaf with a strictly larger score is ever discarded, and the final incumbent attains the maximum over  $\mathcal{F}$ . (With  $\mathcal{E} = \emptyset$ , i.e. no valid range, the code returns an invalid result without searching.)  $\square$

The guarantee holds for pyramid values stored in single precision and summed in double: each per-endpoint term at depth  $h$  is an exact maximum of leaf values, and the endpoints are summed in the same order at every depth, so monotonicity of rounded addition preserves the inequality. The unit tests check it directly: on randomized small cases the branch-and-bound score equals, bit for bit, that of an exhaustive search over  $\mathcal{F}$  (`matchExhaustive`), and three negative controls in the suite confirm the test's power: through a test-only hook (`setBoundMutationForTesting`) that perturbs the coarse-level bounds, an inflated bound (still admissible) keeps the agreement, whereas a deflated bound ( $0.8 S_h$ ) and coarse nodes that use the leaf's drop rule instead of the clamp, the inadmissible bound of the proof's remark, are both detected. It is optimality on the rotation–translation lattice only, not over continuous poses, and for the correlative surrogate  $S_0$ , not for the pose posterior; it is not optimal localization in any statistical sense. In map-sized mode it is *global* over the grid's cell centers whenever the cap  $W_{\max}$  does not bind. The match is accepted only when

$$S_0^* \geq \rho |\mathcal{E}|, \quad (11)$$

**Algorithm 2:** Branch-and-bound global relocalization (bbs . cpp)

---

**Input:** scan endpoints  $\mathcal{E}$ ; pyramid  $\{P_h\}_{h=0}^D$ ; rotations  $\{\theta_k\}$ ; half-windows  $L_x, L_y$ ; feasible set  $\mathcal{F}$ ; acceptance fraction  $\rho$

```

1  $C \leftarrow \{(k, \mathbf{o}, D, S_D(k, \mathbf{o})) : \forall k, o_x \in \{-L_x, -L_x + 2^D, \dots\} \leq L_x, o_y \in \{-L_y, -L_y + 2^D, \dots\} \leq L_y\}$ ;
2  $S^* \leftarrow -1$ ;
3 Procedure Branch( $C$ )
4   sort  $C$  by bound, descending;
5   foreach  $(k, \mathbf{o}, h, U) \in C$  do
6     if  $U \leq S^*$  then break // admissible prune;
7     if  $h = 0$  then
8       if  $(k, \mathbf{o}) \in \mathcal{F}$  then  $S^* \leftarrow U$ ;  $\mathbf{x}^* \leftarrow (\mathbf{c} + \text{res} \cdot \mathbf{o}, \theta_k)$ ;
9     else
10      Branch( $\{(k, \mathbf{o} + \delta, h-1, S_{h-1}(k, \mathbf{o} + \delta)) : \delta \in \{0, 2^{h-1}\}^2, o_x + \delta_x \leq L_x, o_y + \delta_y \leq L_y\}$ );
11 Branch( $C$ );
Output:  $\mathbf{x}^*$ , valid iff  $S^* \geq 0$  and  $S^* \geq \rho|\mathcal{E}|$  (Equation (11))

```

---

with  $\rho = \text{bbs\_min\_score\_fraction}$  and  $|\mathcal{E}|$  the number of valid endpoints used. The search is deterministic and therefore reproducible in a unit test. On maps with repeated structure a single accepted optimum can be a look-alike place (Section 4), so the node does not commit it directly; it verifies it over several scans (Section 3.5).

### 3.5. Verifying a Relocalization over Several Scans

RelocalizationVerifier turns the single-scan decision into a multi-hypothesis one, a simple instance of the established practice of tracking several global-localization hypotheses until later observations separate them (Gao et al., 2019; García et al., 2023). *Hypotheses.* On the first scan the branch-and-bound search keeps up to  $K = \text{bbs\_verify\_top\_k}$  distinct modes instead of one: two leaves belong to the same mode when they lie within  $\text{bbs\_verify\_nms\_xy}$  (1 m) and  $\text{bbs\_verify\_nms\_yaw}$  (0.35 rad) of each other, and only the higher-scoring one is kept (greedy non-maximum suppression). Pruning uses the  $K$ -th kept score in place of  $S^*$ , so the first mode is still the exact optimum of Corollary 1; the lower modes are the best non-suppressed leaves, without that guarantee, because greedy suppression depends on the visiting order. Every mode that passes Equation (11) becomes a hypothesis  $i$  with a uniform prior. *Evidence.* On each later scan every hypothesis is moved by the odometry increment and re-matched by a small branch-and-bound search ( $\pm \text{bbs\_verify\_track\_linear\_window} = 0.3 \text{ m}, \pm 0.1 \text{ rad}$ , exact ties resolved toward the prediction). With  $f_i^t = S_0/|\mathcal{E}|$  the per-beam score of that re-match, the log-weights and the normalized evidence score  $\pi_i$  are

$$\ell_i = \lambda \sum_{t=1}^M f_i^t, \quad \pi_i = \frac{\exp \ell_i}{\sum_{j \text{ alive}} \exp \ell_j}, \quad (12)$$

with  $\lambda = \text{bbs\_verify\_evidence\_gain}$ ; a hypothesis whose re-match window holds no feasible pose dies. *Decision.* After  $M = \text{bbs\_verify\_scans}$  scans (the first included) the best hypothesis is committed only if  $\pi_{i^*} \geq \tau = \text{bbs\_verify\_min\_posterior}$  and its mean  $f$  passes  $\rho$ ; if no hypothesis reaches  $\tau$ , the node logs the place as *ambiguous*, does not touch the particle set, and retries from the next scan.  $\lambda$  is a temperature: Equation (12) treats per-scan scores as independent evidence, which beams are not, so  $\pi$  is a normalized heuristic evidence (confidence) score, not a Bayesian

posterior and not a calibrated probability; its gain was selected for few false accepts on a separate run (below), not so that  $\pi$  matches observed error rates. The code keeps the name *posterior* for  $\pi_i$  (RelocHypothesis::posterior, bestPosterior(), and the CSV column best\_posterior) and for its threshold  $\tau$  (min\_posterior, ROS parameter bbs\_verify\_min\_posterior); the node's log messages and PARAMS.md also use the word *posterior*, and PARAMS.md calls the value a probability. In this paper these identifiers and words all denote the evidence score and its threshold, not a calibrated probability. While verification runs, an existing particle set keeps tracking unchanged; a committed pose re-seeds the filter with a Gaussian of standard deviation (0.2 m, 0.2 m, 0.1 rad). With  $K = M = 1$  the rule reduces exactly to the single-scan test (a unit test checks this). The defaults  $K = 8$ ,  $M = 9$ ,  $\lambda = 40$ ,  $\tau = 0.9$  were chosen on a separate parameter-selection run (called calib in the code) whose maps and poses are disjoint from the study of Section 4 (seed 20261105): experiments/select\_verify\_params.py replays the decision rule over  $K \in \{1, 2, 3, 4, 6, 8\}$ ,  $M \in \{1, \dots, 12\}$ ,  $\lambda \in \{10, 20, 40, 80, 160\}$ ,  $\tau \in \{0.9, 0.95, 0.99, 0.999\}$  and takes the configuration with the fewest false accepts, then the most correct accepts. On the 200 parameter-selection queries it gives 163 correct accepts and 0 false accepts (9 ambiguous), against 160 and 12 for the single-scan rule.

### 3.6. Fusing IMU, LiDAR, and GNSS: A 15-State Error-State Kalman Filter

The fusion3d backend runs ESKF, which follows the quaternion error-state formulation of Solà (2017). The nominal state carries position  $\mathbf{p}$ , velocity  $\mathbf{v}$ , orientation  $\mathbf{q}$ , and accelerometer and gyroscope biases  $\mathbf{b}_a, \mathbf{b}_g$ ; the error state is  $\delta\mathbf{x} = (\delta\mathbf{p}, \delta\mathbf{v}, \delta\theta, \delta\mathbf{b}_a, \delta\mathbf{b}_g) \in \mathbb{R}^{15}$  with a local angular error  $\delta\theta$ .

**Prediction.** On each IMU sample  $(\mathbf{a}_m, \omega_m)$  with period  $\Delta t$ , let  $\mathbf{R} = \mathbf{R}(\mathbf{q})$ ,  $\mathbf{a} = \mathbf{a}_m - \mathbf{b}_a$ ,  $\omega = \omega_m - \mathbf{b}_g$ , and  $\mathbf{a}_w = \mathbf{R}\mathbf{a} + \mathbf{g}$  with  $\mathbf{g} = (0, 0, -9.81)$ . The nominal state and error covariance evolve as

$$\mathbf{p} += \mathbf{v}\Delta t + \frac{1}{2}\mathbf{a}_w\Delta t^2, \quad \mathbf{v} += \mathbf{a}_w\Delta t, \quad \mathbf{q} \leftarrow \mathbf{q} \otimes \text{Exp}(\omega\Delta t), \quad \mathbf{P} \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}, \quad (13)$$

where  $\text{Exp}$  is the  $SO(3)$  exponential,  $\mathbf{F}$  is the  $15 \times 15$  identity except for the blocks

$$\mathbf{F}_{pv} = \mathbf{I}\Delta t, \quad \mathbf{F}_{v\theta} = -\mathbf{R}[\mathbf{a}]_\times\Delta t, \quad \mathbf{F}_{vb_a} = -\mathbf{R}\Delta t, \quad \mathbf{F}_{\theta\theta} = \mathbf{R}\{\text{Exp}(\omega\Delta t)\}^\top, \quad \mathbf{F}_{\theta b_g} = -\mathbf{I}\Delta t,$$

and the process noise

$$\mathbf{Q} = \text{blkdiag}(\mathbf{0}_3, \sigma_a^2\Delta t^2\mathbf{I}_3, \sigma_g^2\Delta t^2\mathbf{I}_3, \sigma_{b_a}^2\Delta t\mathbf{I}_3, \sigma_{b_g}^2\Delta t\mathbf{I}_3)$$

is built from sigma\_acc, sigma\_gyro, sigma\_acc\_bias, and sigma\_gyro\_bias;  $\mathbf{R}\{\cdot\}$  denotes the rotation matrix of a quaternion.

**Correction, Injection, Reset.** Two updates share the Kalman gain,

$$\mathbf{K} = \mathbf{P}\mathbf{H}^\top(\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R}_z)^{-1}, \quad \delta\hat{\mathbf{x}} = \mathbf{K}\mathbf{y}, \quad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}. \quad (14)$$

updatePose applies a 6-DoF pose from NDT registration with  $\mathbf{H}$  selecting  $(\delta\mathbf{p}, \delta\theta)$  and residual  $\mathbf{y} = (\mathbf{p}_m - \mathbf{p}, \text{Log}(\mathbf{q}^{-1} \otimes \mathbf{q}_m))$ , with  $\mathbf{R}_z = \text{diag}(\sigma_{\text{pos}}^2\mathbf{I}, \sigma_{\text{rot}}^2\mathbf{I})$  (pose\_pos\_std, pose\_rot\_std); the node applies it only when PCL's NDT converged with fitness at most ndt\_max\_fitness. updatePosition applies a 3-DoF GNSS fix, converted from WGS-84 through ECEF to a local ENU frame about a datum (the first accepted fix unless use\_datum fixes one), with  $\mathbf{H}$  selecting  $\delta\mathbf{p}$  and  $\mathbf{R}_z$  taken from the fix covariance (horizontal variances floored at  $10^{-4}$ , vertical at 1). After each update the error is injected, additively for  $\mathbf{p}, \mathbf{v}, \mathbf{b}_a, \mathbf{b}_g$  and as  $\mathbf{q} \leftarrow \mathbf{q} \otimes \text{Exp}(\delta\hat{\theta})$  for orientation, and the error mean is reset to zero. The implementation uses the simple covariance update of Equation (14). By default it omits the reset Jacobian (it takes  $\mathbf{G} = \mathbf{I}$ ), a common first-order simplification of the full formulation in Solà (2017); the option reset\_jacobian applies  $\mathbf{P} \leftarrow \mathbf{G}\mathbf{P}\mathbf{G}^\top$  with  $\mathbf{G} = \text{blkdiag}(\mathbf{I}, \mathbf{I}, \mathbf{I} - [\frac{1}{2}\delta\hat{\theta}]_\times, \mathbf{I}, \mathbf{I})$  after each injection, and Section 4.3 measures its effect on consistency.

### 3.7. Initializing and Diagnosing: Making Every Precondition Observable

All backends accept a `PoseWithCovarianceStamped` on `/initialpose`: the MCL backends spread a Gaussian particle cloud at the supplied mean and covariance, and the fusion backend loads it as its nominal state. Automatically, `laser2d` runs Algorithm 2 before its first pose when `try_global_localization` is set, or on demand through the `~/global_localization` service (the matcher is constructed lazily at the first call). `fusion3d` auto-initializes once it holds both an attitude, roll and pitch from the first IMU specific force  $\mathbf{f}$  and heading  $\psi_0$  from `initial_yaw`,

$$\phi_0 = \text{atan2}(f_y, f_z), \quad \vartheta_0 = \text{atan2}(-f_x, \sqrt{f_y^2 + f_z^2}), \quad \mathbf{q}_0 = \mathbf{R}_z(\psi_0) \mathbf{R}_y(\vartheta_0) \mathbf{R}_x(\phi_0), \quad (15)$$

and a position from the first GNSS fix that passes the gate  $\{\text{status} \geq s_{\min}\} \wedge \{\text{lat, lon not NaN}\} \wedge \{c_{xx} \leq c_{\max}\}$  (`gnss_min_status`, `gnss_max_pos_cov`).

Because a localization node that silently does nothing is the dominant first-run frustration, four diagnostic mechanisms accompany the estimators: (i) `ndt3d` and `fusion3d` abort construction with an actionable error when `map_pcd_path` is empty or fails to load; (ii) every callback that cannot yet act emits a rate-limited warning naming the unmet precondition and the remedy; (iii) a 10 s startup watchdog reports each silent mandatory input together with its live publisher count, separating “no publisher” from “publisher present but QoS-incompatible”; and (iv) the GNSS gate and the NDT update log each rejection with the measured value against the violated threshold. Every tunable is a declared ROS 2 parameter documented one-to-one in `PARAMS.md` (Section B). These mechanisms carry no algorithmic weight; they exist so that a misconfiguration is diagnosable from the console log alone, in the spirit of the logging infrastructure Nav2 relied on in its long-duration field experiment (Macenski et al., 2020).

## 4. Experiments

We validate PRISM-Loc at the level of *correctness and closed-loop behavior*, not field accuracy. Section 4.1 describes the test suite and the six synthetic closed-loop experiments, Section 4.2 reports their results, and Section 4.3 isolates the contribution of individual mechanisms using the same committed runs. All data in this section are synthetic, and we make no accuracy claim about physical robots.

### 4.1. Experimental Setup

**Correctness Tests.** The suite comprises 63 GoogleTest cases across the four packages: 42 in `prism_loc_core` (both MCL backends and global recovery), 15 in `prism_loc_fusion` (the ESKF), 4 in the `prism_loc` adapter layer, and 2 in `prism_loc_fusion_ros`. The motion model is checked deterministically (zero-command and pure-translation cases) and statistically (the mean of noisy samples approximates the commanded motion); both measurement models are checked for the monotonicity a likelihood must satisfy (the true pose out-scores a displaced one); resampling, the weighted-mean estimate, the grid and NDT-map primitives, and branch-and-bound recovery of a single-scan kidnapped pose (with rejection of an empty scan) are tested directly, the latter also on a 40 m×30 m map with the pose 17 m from the center, together with the symmetric window, the compute cap, the rejection of occupied and off-map candidates, and bit-exact agreement with exhaustive search (Corollary 1) with three negative controls that perturb the bound; the top- $K$  search and the verifier of Section 3.5 are tested on twin rooms that are identical within sensor range: the first mode equals the single-best match, both twins are returned with equal score, a scan that fools the single-scan

rule (an unmapped box beside the robot matches a mapped box in the other twin) is resolved once driving brings a distinguishing pillar into range, a place that nothing in range distinguishes is reported ambiguous rather than committed, an unambiguous place is accepted at the true pose, and  $K = M = 1$  reproduces the single-scan result; two integration tests close the `laser2d` and `ndt3d` loops until convergence to ground truth. The ESKF is pinned for rest under a static IMU, constant-acceleration kinematics, measurement pull-in, and tracking of a moving target from a wrong initial guess;  $SO(3)$  exponential/logarithm round trips, geodetic ENU round trips, and the Equation (2) composition are verified separately, and RNG tests assert that equal seeds reproduce identical sequences; two tests pin the optional ESKF reset Jacobian. Two CI jobs guard the boundary from both sides: a `core` job builds the two cores with plain CMake and runs `cctest` with only a compiler and Eigen, and a `ROS` job runs `colcon build` and `colcon test` for all four packages in a `ros:humble` container.

**Synthetic Closed-Loop Protocol.** Six drivers under `experiments/` compile the *actual* core sources directly with `g++` and Eigen (no ROS, no CMake) and generate their own worlds, scans, and inertial streams, with all randomness derived from fixed compile-time seeds. The sensor-noise models match the filters' assumptions by construction, so the experiments demonstrate behavior rather than benchmark accuracy.

- *MCL tracking.* A  $12\text{ m} \times 10\text{ m}$  grid at  $0.1\text{ m/cell}$  (room border plus two interior pillars) hosts a robot driven around a closed oval loop with semi-axes  $3.6\text{ m}$  and  $2.6\text{ m}$  for  $65\text{ s}$  at  $10\text{ Hz}$  ( $650$  steps). Each step ray-casts a  $180$ -beam scan at the true pose and synthesizes drifting odometry from noise-corrupted increments under the `laser2d.yaml` model ( $\alpha_{1..4} = 0.2$ ). The filter, with every parameter from `laser2d.yaml`, is seeded with a broad Gaussian prior ( $\sigma = 1\text{ m}, 0.4\text{ rad}$ ) at the true start and updated as in Algorithm 1 with  $d_{\min} = a_{\min} = 0.2$ . Seeds: 101, 202, 303.
- *BBS relocation.* On an asymmetric  $6\text{ m} \times 6\text{ m}$  room (two interior wall stubs break the four-fold symmetry, the world of the BBS unit test),  $N = 100$  poses are drawn uniformly over free space with uniform yaw, and one  $180$ -beam scan is ray-cast per pose. To span easy to hard cases, each beam is replaced with a uniform-random spurious range with a per-query probability drawn uniformly in  $[0, 0.8]$ , modeling people, glass, and objects absent from the map. The real matcher runs with the `bbs_*` defaults ( $W = 10\text{ m}$ ,  $\pm\pi$  at  $0.0175\text{ rad} \approx 1^\circ$ ,  $D = 6$ , `bbs_max_beams` =  $120$ , which for a  $180$ -beam scan gives stride  $1$ , so all  $180$  beams are scored), centered on the room, so its window covers the whole map. The committed CSV was produced by the first release of the matcher; re-running the driver against the current matcher (symmetric window, feasibility rule) reproduces every column except wall-clock latency. Seed: 20240607.
- *ESKF under aiding dropout.* The real `Eskf` runs over a  $60\text{ s}$  3D trajectory, a horizontal figure-eight  $x = 8 \sin \omega t$ ,  $y = 4 \sin 2\omega t$  with  $\omega = 2\pi/30$  and a  $\pm 1.5\text{ m}$  altitude oscillation. The analytic specific force and angular rate are corrupted with exactly the white-noise densities and constant biases the process model assumes (`fusion3d.yaml` defaults), plus fixed true biases the filter must estimate from zero. The IMU runs at  $200\text{ Hz}$ , a simulated NDT source delivers 6-DoF poses at  $10\text{ Hz}$  and a simulated GNSS source positions at  $1\text{ Hz}$ , with noise matched to the filter's  $\mathbf{R}_z$ ; both aiding streams are withheld for  $t \in [25, 35]\text{ s}$ . Seed: 20240517.
- *BBS on large maps.* Square warehouse-like grids of side  $20, 40$ , and  $80\text{ m}$  at  $0.1\text{ m/cell}$  carry border walls, *repeated* shelf rows ( $0.5\text{ m}$  deep at a  $6\text{ m}$  pitch with  $1$ – $2$  random gaps each) and random  $0.4\text{ m}$  pillars; the repetition deliberately creates look-alike places. Kidnapped poses are drawn uniformly over free space and stratified into  $60$  inside and  $60$  outside the set that the first release searched ( $-10\text{ m}$  to  $+15.5\text{ m}$  about the map center; the  $20\text{ m}$  map lies entirely inside it). One  $180$ -beam,  $30\text{ m}$ -range scan per pose receives the room study's clutter model, and the first-release matcher (a verbatim copy, centered on the map midpoint with  $W = 10\text{ m}$ ) and the current map-sized matcher process the identical scan. Seed: 20260928.

**Table 3: laser2d steady-state tracking error on synthetic data** ( $t \geq 10$  s, 110 logged samples per seed at 2 Hz). Columns are computed from `data/mcl_tracking.csv`; the last two rows pool the three seeds as mean  $\pm$  std of the per-seed values and as statistics over all 330 samples.

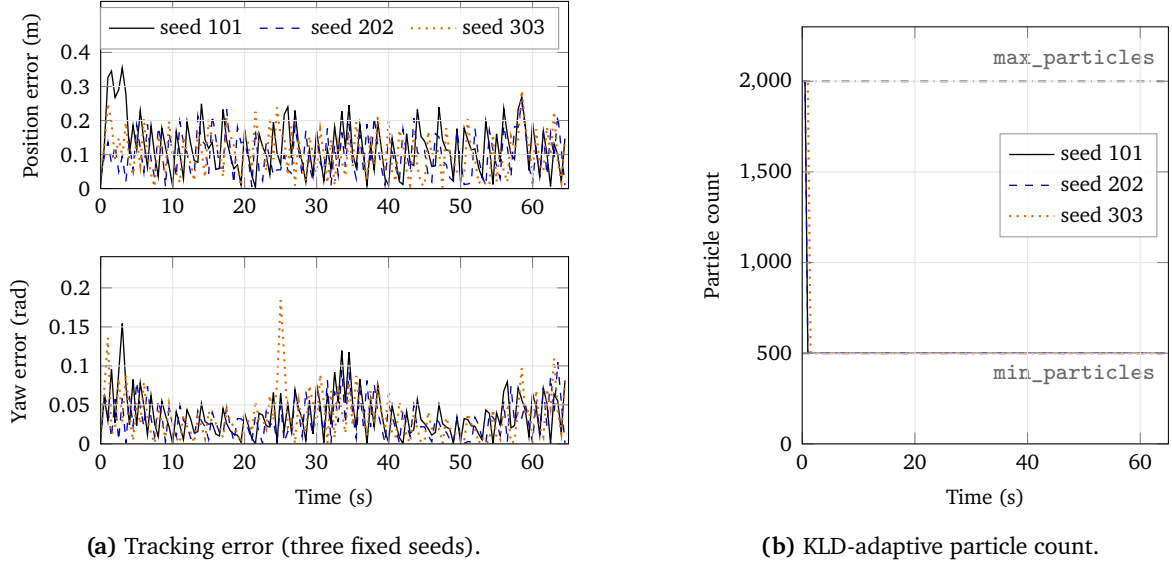
| Seed                    | Mean pos. err. (m) | Max pos. err. (m) | Mean yaw err. (rad) | Max yaw err. (rad) |
|-------------------------|--------------------|-------------------|---------------------|--------------------|
| 101                     | 0.113              | 0.269             | 0.034               | 0.120              |
| 202                     | 0.100              | 0.257             | 0.030               | 0.105              |
| 303                     | 0.102              | 0.288             | 0.033               | 0.184              |
| Per-seed mean $\pm$ std | 0.105 $\pm$ 0.007  | –                 | 0.032 $\pm$ 0.002   | –                  |
| Pooled (330 samples)    | 0.105              | 0.29              | 0.032               | 0.18               |

- *Multi-scan verification.* On the maps, poses, and first scans of the large-map study (same seed and draw order, so the single-scan rows reproduce it exactly), the robot then drives a short random trajectory: 0.3 m forward steps with  $\mathcal{N}(0, 0.15 \text{ rad})$  heading changes, an in-place turn when a step would leave free space, and one scan per step with the query’s clutter fraction. The verifier receives odometry increments corrupted by  $\sigma = 0.02|d| + 0.005 \text{ m}$  per axis and  $0.01 + 0.02|\Delta\theta| \text{ rad}$ . Success is judged against the true pose at the scan where the method commits. Seed: 20260928 (study); 20261105 for the parameter-selection run of Section 3.5.
- *ESKF Monte Carlo NEES.*  $N = 100$  runs of the ESKF scenario above in which the initial position, velocity, and attitude errors and the true IMU biases are drawn from the filter’s prior  $\mathcal{N}(\mathbf{0}, \mathbf{P}_0)$ , and the true biases then random-walk with the filter’s bias process noise, so that the prior, the sensor and bias noise, and the aiding noise all match the filter’s model; the one unmodeled effect is that the IMU samples an analytic trajectory, whose integration between samples the zero-order-hold process model does not represent exactly. Every 0.5 s we average the normalized estimation error squared  $\epsilon = \mathbf{e}^\top \mathbf{P}^{-1} \mathbf{e}$  over the runs, for the full 15-D error and for each 3-D block, and compare it with the two-sided 95% interval  $[\chi_{0.025}^2(Nn)/N, \chi_{0.975}^2(Nn)/N]$  for  $n$  degrees of freedom (Bar-Shalom et al., 2001). The shipped filter and the reset-Jacobian variant process identical data. No parameter was tuned on these runs. Seeds: 20260929 plus the run index, and 20261029 plus the run index for the bias random walk.

**Baselines, Metrics, and Hardware.** No external baseline is run: neither a tracker such as `nav2_amcl`, Beluga (Puga et al., 2024), MH-AMCL (García et al., 2023), or `hdl_localization` (Koide et al., 2019), nor a global localizer such as `hdl_global_localization` (Koide, 2021) or CBGL (Filotheou, 2024), nor another multi-hypothesis verifier (Gao et al., 2019). The synthetic worlds are designed to exercise PRISM-Loc’s own cores, every comparison below is between configurations of PRISM-Loc itself, and a comparative study on real and public data is left to future work (Section 5). We report position error (Euclidean, m) and absolute yaw error against synthetic ground truth, BBS success rate (position error  $< 0.5 \text{ m}$  and yaw error  $< 10^\circ$ ), acceptance and false-accept counts at the threshold  $\rho$  of Equation (11), windowed RMSE, the  $3\sigma$  envelope, and Monte Carlo NEES for the ESKF, and BBS wall-clock latency. All accuracy numbers are deterministic; latency is machine-dependent and was measured single-threaded on the machine that produced the committed CSV.

## 4.2. Main Results

**MCL Tracking and KLD Adaptation.** Figure 2 and Table 3 show that after convergence ( $t \geq 10$  s, across all three seeds) the `laser2d` filter holds a mean position error of 0.105 m (max 0.29 m) and a mean yaw error of 0.032 rad, i.e.  $1.8^\circ$  (max 0.18 rad), with little spread across seeds (0.105  $\pm$  0.007 m over per-seed means). The KLD bound (Equation (5)) sizes the set to the posterior: it starts at



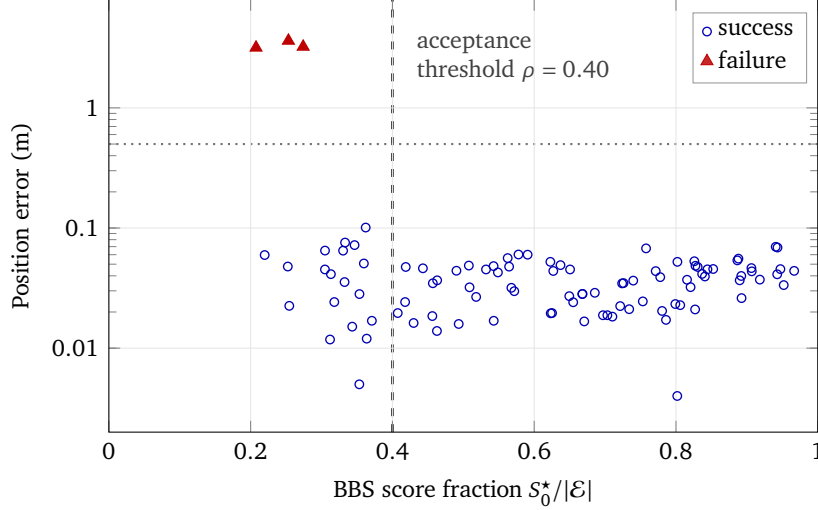
**Figure 2: laser2d closed-loop tracking on synthetic data.** A simulated robot drives a closed oval loop (semi-axes 3.6 m and 2.6 m) for 65 s at 10 Hz in a 12 m×10 m grid with two symmetry-breaking pillars; the real ParticleFilter with Laser2DLikelihoodField and KLD resampling is updated exactly as in Algorithm 1. (a) Starting from a broad prior, the filter converges within a few updates; across the three seeds the steady-state ( $t \geq 10$  s) mean position error is 0.105 m (max 0.29 m) and the mean yaw error 0.032 rad (max 0.18 rad). (b) The particle set starts at the 2000-particle ceiling and collapses to the 500-particle floor after the first post-convergence update, where it stays.

the 2000-particle ceiling under the broad prior, collapses to the 500-particle floor after the first post-convergence update, and holds there for the rest of the loop, a 4× reduction in per-update likelihood evaluations once the posterior is concentrated.

**Global Relocalization.** Of the 100 kidnapped-robot queries, 97 succeed (Table 4); over all queries the recovered pose has median position error 0.037 m (95th percentile 0.072 m) and median yaw error  $0.48^\circ$  (95th percentile  $1.74^\circ$ ), consistent with the 0.1 m grid and  $\approx 1^\circ$  rotation lattice. Figure 3 shows that the score fraction of Equation (11) is an informative confidence signal: the three failures score in 0.21–0.27 with more than 3 m and roughly  $90^\circ$ – $180^\circ$  error, all below  $\rho = 0.40$ , so the default threshold yields *zero false accepts* while accepting 78 matches, all correct, and conservatively rejecting 19 correct fixes. The median score fraction is 0.632 and successes span 0.22–0.97, so three low-scoring successes overlap the failure band. Median full-map query latency is 37.65 ms.

**ESKF Fusion.** With full aiding the windowed position RMSE is 0.016 m (Table 4). During the 10 s blackout the filter dead-reckons on the IMU alone and the RMSE rises to 0.125 m, with a peak error of 0.256 m at  $t = 34.5$  s at the end of the blackout; the first aiding cycle brings the error back to 0.015 m by  $t = 35$  s, and the post-dropout RMSE is 0.009 m (Figure 4). The filter’s own  $3\sigma$  bound inflates from roughly 0.03–0.04 m under full aiding to 0.663 m and stays above the true error at every logged sample, i.e. the reported position covariance is not overconfident on this single seeded run. This is a one-sided envelope check on one run; the NEES study below is the consistency test.

**Relocalization on Large Maps.** Table 5 moves the kidnapped-robot study to maps larger than the room. For poses outside the set the first release searched, that matcher succeeds on only 2 of 60 queries on the 40 m map and 0 of 60 on the 80 m map, and at  $\rho = 0.40$  it *accepts* a wrong in-window pose on 39 and 51 of them; the map-sized matcher recovers 50 and 41 of the same 60 poses. Inside that set the two matchers perform alike, as they should. The map-sized window does not, however, make acceptance safe. At  $\rho = 0.40$  the current matcher accepts a wrong pose on 6 of 120 queries on



**Figure 3: Branch-and-bound relocalization of a kidnapped robot on synthetic data.** Each marker is one of  $N = 100$  queries in an asymmetric  $6\text{ m} \times 6\text{ m}$  room: position error versus the score fraction of Equation (11). Each 180-beam scan is corrupted with a per-query clutter probability drawn uniformly in  $[0, 0.8]$ . Success means position error  $< 0.5\text{ m}$  (dotted) and yaw error  $< 10^\circ$ . All three failures score below the default threshold (dashed), so every accepted match is correct (zero false accepts); 19 correct fixes are conservatively rejected.

the 40 m map and 15 of 120 on the 80 m map (first release: 43 and 57). Of these 21 false accepts, 13 are  $180^\circ$  flips and none is closer than 4.95 m to the truth; they are look-alike aisles, not near misses. No threshold removes them cheaply: the highest-scoring false match on the two large maps has score fraction 0.71, and only 94 of the 198 successes score above it. The feasibility rule does act: the first release returned a pose inside an obstacle or off the map on 8 of its 300 queries, the current matcher on none. Global coverage also costs time: the median query takes 29 ms on the 20 m map but 1526 ms on the 80 m map, with a 95th percentile of 6374 ms under heavy clutter.

**Verification against Aliasing.** Table 6 replaces the single-scan acceptance with the verifier of Section 3.5 on the same queries. False accepts fall from 6 of 120 to 0 of 120 on the 40 m map and from 15 to 1 of 120 on the 80 m map; pooled over both maps the false-accept rate falls from 21 of 240 [5.8, 13.0] % to 1 of 240 [0.1, 2.3] % (95% Wilson score intervals, Wilson 1927). Of the 21 single-scan false accepts, 9 become correct fixes, 8 end as ambiguous, and 3 as no candidate; 1 remains a false accept. That residual case is a  $180^\circ$  flip on the 80 m map, 34.7 m from the truth, which held 0.969 of the evidence score after nine scans: verification makes a confident wrong pose rarer, it does not rule it out. The price is small in correct fixes and moderate in time. The verifier commits 242 correct poses against 241 for the single scan over all 300 queries; it loses 8 fixes the single scan had (2 ambiguous, 6 rejected because the mean score fraction over the nine scans falls below  $\rho$ , these six all at clutter fractions of at least 0.66) and gains 9. Its median decision takes 1622 ms on the 80 m map against 1247 ms for the single scan in the same run (three concurrent workers, so not comparable with the single-threaded latencies above), dominated by the wider top-8 search, and it needs the robot to move for eight more scans; a stationary robot in a look-alike place stays ambiguous by design.

**ESKF Consistency.** The NEES test gives a negative verdict (Table 7, Figure 5): the filter is overconfident. Its time-averaged full-state NEES is 18.05 against 15 degrees of freedom, and at single epochs the run average lies inside the 95% interval at only 31 of 120 epochs and above it (overconfident) at the other 89; it is never below it. The blocks locate the problem. Attitude is the worst block (above at 47 epochs, peak 5.91 against an upper limit of 3.50); position (inside at 104 of 120 epochs) and velocity are overconfident mainly during and just after the aiding dropout; both bias blocks are consistent

**Table 4: Relocalization and fusion results on synthetic data.** Left: branch-and-bound over  $N = 100$  kidnapped-robot queries (data/bbs\_relocalization.csv); percentiles use linear interpolation; latency is single-threaded wall-clock on the machine that produced the committed CSV. Right: ESKF position error by window relative to the 10 s aiding dropout (data/eskf\_track.csv, logged at 2 Hz).

| <b>BBS relocalization</b>                   |                             |
|---------------------------------------------|-----------------------------|
| Success (pos. $< 0.5$ m, yaw $< 10^\circ$ ) | 97 / 100                    |
| Median / p95 position error                 | 0.037 / 0.072 m             |
| Median / p95 yaw error                      | $0.48^\circ$ / $1.74^\circ$ |
| Accepted at $\rho = 0.40$ / false accepts   | 78 / 0                      |
| Median score fraction                       | 0.632                       |
| Median latency (min, p95)                   | 37.65 ms (8.77, 106.07)     |
| <b>ESKF fusion</b>                          |                             |
| RMSE before dropout ( $1 \leq t < 25$ s)    | 0.016 m                     |
| RMSE during dropout ( $25 \leq t < 35$ s)   | 0.125 m                     |
| RMSE after dropout ( $t \geq 35$ s)         | 0.009 m                     |
| Peak error (at $t = 34.5$ s)                | 0.256 m                     |
| Peak $3\sigma$ bound (at $t = 34.5$ s)      | 0.663 m                     |
| Samples with error $> 3\sigma$              | 0 / 121                     |

**Table 5: Relocalization on large synthetic maps: v0.1 window versus map-sized window.** Warehouse-like maps with repeated shelf rows at 0.1 m/cell; kidnapped poses stratified by whether they lie inside (in) or outside (out) the set the v0.1 node searched ( $-10$  m to  $+15.5$  m about the map center); the 20 m map lies entirely inside it. Each cell reads v0.1 / current matcher on identical scans ( $U[0, 0.8]$  beam clutter). Success: position error  $< 0.5$  m and yaw error  $< 10^\circ$ ; accepted:  $S_0^* \geq 0.40|\mathcal{E}|$ ; false accept: accepted but not a success. Latency is the single-threaded median (data/bbs\_largemap.csv, generated by experiments/bbs\_largemap.cpp).

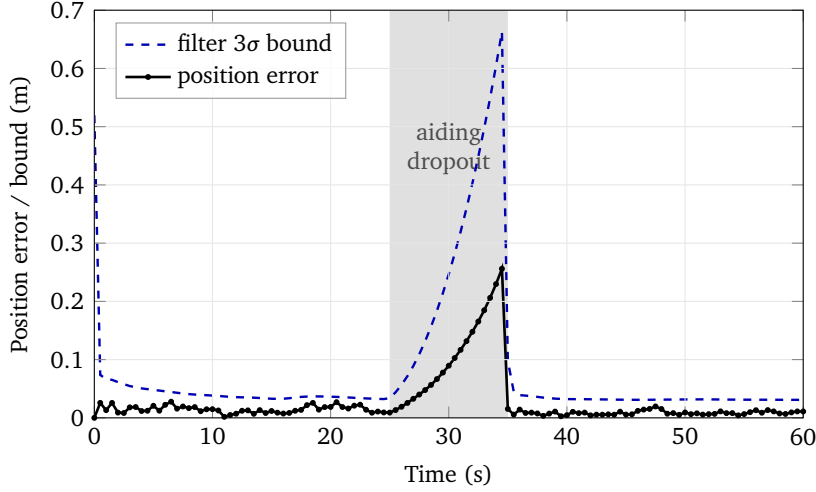
| Map  | Poses | $N$ | Success | Accepted | False accepts | Median latency (ms) |
|------|-------|-----|---------|----------|---------------|---------------------|
| 20 m | in    | 60  | 60 / 60 | 50 / 50  | 0 / 0         | 29 / 29             |
| 40 m | in    | 60  | 53 / 54 | 57 / 57  | 4 / 3         | 74 / 127            |
| 40 m | out   | 60  | 2 / 50  | 41 / 48  | 39 / 3        | 150 / 220           |
| 80 m | in    | 60  | 51 / 53 | 54 / 55  | 6 / 4         | 391 / 1547          |
| 80 m | out   | 60  | 0 / 41  | 51 / 52  | 51 / 11       | 334 / 1484          |

(inside at 116 and 120 of 120 epochs). The published orientation covariance is therefore too small, and the position covariance is too small while aiding is absent. A first version of this study held the true biases constant while the filter injects bias random-walk noise; with that mismatch both bias blocks read as underconfident, and with the bias random walk added to the truth they are consistent.

### 4.3. Ablation Studies

The committed runs also isolate the effect of four mechanisms. All numbers below are recomputed from the same CSVs; no additional experiment was run for this section.

**Acceptance Threshold.** Table 8 sweeps  $\rho$  over the recorded score fractions. Without the gate ( $\rho \leq 0.20$ ) all three failures would re-seed the filter at a wrong pose; any  $\rho \in [0.30, 0.60]$  removes every false accept, and the shipped  $\rho = 0.40$  trades 19 rejected correct fixes for a margin of 0.13 above the highest-scoring failure. Because a rejected match merely waits for the next scan, whereas a false accept silently corrupts the posterior, we keep the conservative default;  $\rho = 0.30$  would have accepted 16 more correct fixes on this data.



**Figure 4: ESKF fusion under a 10 s aiding dropout on synthetic data.** A 200 Hz IMU is fused with 10 Hz NDT-like 6-DoF pose and 1 Hz GNSS-like position updates over a 60 s 3D figure-eight; both aiding streams are withheld for  $t \in [25, 35]$  s (shaded). The position error grows to its peak of 0.256 m on dead reckoning alone while the filter’s own  $3\sigma$  bound inflates to 0.663 m, and both collapse within one aiding cycle once updates resume. The bound stays above the true error at every logged sample.

**Table 6: Single-scan acceptance versus multi-scan verification** on the maps, poses and first scans of Table 5. Each cell reads single scan / verified ( $K = 8$  modes,  $M = 9$  scans, Section 3.5); the robot drives 0.3 m steps with noisy odometry between scans. Correct: accepted and within 0.5 m and  $10^\circ$  of the truth at the committed scan; false accepts with the 95% Wilson interval of their rate in percent; ambiguous: verification ended without a hypothesis holding 0.9 of the normalized evidence score. Latency: median over the whole decision, measured with three concurrent workers (data/bbs\_verify.csv, experiments/bbs\_verify.cpp).

| Map       | Poses | $N$ | Correct   | False accepts [95% CI, %]      | Ambiguous | Median latency (ms) |
|-----------|-------|-----|-----------|--------------------------------|-----------|---------------------|
| 20 m      | in    | 60  | 50 / 49   | 0 [0.0, 6.0] / 0 [0.0, 6.0]    | 0         | 35 / 127            |
| 40 m      | in    | 60  | 54 / 54   | 3 [1.7, 13.7] / 0 [0.0, 6.0]   | 1         | 129 / 261           |
| 40 m      | out   | 60  | 45 / 45   | 3 [1.7, 13.7] / 0 [0.0, 6.0]   | 1         | 238 / 399           |
| 80 m      | in    | 60  | 51 / 50   | 4 [2.6, 15.9] / 0 [0.0, 6.0]   | 2         | 1351 / 1752         |
| 80 m      | out   | 60  | 41 / 44   | 11 [10.6, 29.9] / 1 [0.3, 8.9] | 6         | 1093 / 1526         |
| 40 + 80 m | all   | 240 | 191 / 193 | 21 [5.8, 13.0] / 1 [0.1, 2.3]  | 10        | 535 / 762           |

**Clutter.** Table 9 shows how clutter affects the matcher. Below a clutter probability of 0.533 every query succeeds and is accepted; in the top third the pose is still recovered in 30 of 33 cases with an unchanged median error, but the median score fraction falls to 0.353, so only 11 are accepted. Clutter therefore degrades the matcher’s *confidence* much faster than its *accuracy*, and the gate absorbs the difference; all three failures occur in this top third.

**KLD Adaptation and Aiding.** Replacing KLD adaptation with a fixed set at the 2000-particle ceiling would cost four times the steady-state likelihood evaluations of the 500-particle floor; averaged over each whole run, including the initial transient, the adaptive set holds 526.9 particles,  $3.8\times$  fewer than the ceiling, with the accuracy of Table 3. The dropout window acts as an ablation of both aiding streams: removing NDT and GNSS updates raises position RMSE from 0.016 m to 0.125 m over 10 s, and restoring them recovers 0.009 m, so the growth is the bounded drift of IMU-only integration from which the filter recovers within one aiding cycle.

**Reset Jacobian.** Enabling `reset_jacobian` on identical data changes the run-averaged NEES of

**Table 7: Monte Carlo NEES of the ESKF** ( $N = 100$  runs, 120 epochs at 2 Hz after  $t = 0$ ; initial error and true IMU biases drawn from  $\mathbf{P}_0$ ). Band: two-sided 95% interval for the run-averaged NEES; a consistent filter keeps about 95% of epochs inside it. In / above / below count epochs for the shipped filter ( $\mathbf{G} = \mathbf{I}$ ); the last column is the time-averaged NEES with the reset Jacobian enabled on identical data (data/eskf\_nees.csv, experiments/eskf\_nees.cpp).

| Error block (dof)                         | 95% band       | Mean NEES, $\mathbf{G} = \mathbf{I}$ | In / above / below | Mean NEES, reset $\mathbf{G}$ |
|-------------------------------------------|----------------|--------------------------------------|--------------------|-------------------------------|
| Full state (15)                           | [13.95, 16.09] | 18.05                                | 31 / 89 / 0        | 18.04                         |
| Position $\delta \mathbf{p}$ (3)          | [2.54, 3.50]   | 3.10                                 | 104 / 15 / 1       | 3.10                          |
| Velocity $\delta \mathbf{v}$ (3)          | [2.54, 3.50]   | 3.15                                 | 94 / 25 / 1        | 3.15                          |
| Attitude $\delta \boldsymbol{\theta}$ (3) | [2.54, 3.50]   | 3.48                                 | 73 / 47 / 0        | 3.48                          |
| Accel. bias $\delta \mathbf{b}_a$ (3)     | [2.54, 3.50]   | 3.09                                 | 116 / 4 / 0        | 3.09                          |
| Gyro bias $\delta \mathbf{b}_g$ (3)       | [2.54, 3.50]   | 3.12                                 | 120 / 0 / 0        | 3.12                          |

**Table 8: Ablation of the BBS acceptance threshold  $\rho$**  (Equation (11)), recomputed post hoc from the score fractions in data/bbs\_relocalization.csv; no search is re-run. “Rejected correct” counts successful matches that the threshold would discard. The shipped default is  $\rho = 0.40$ .

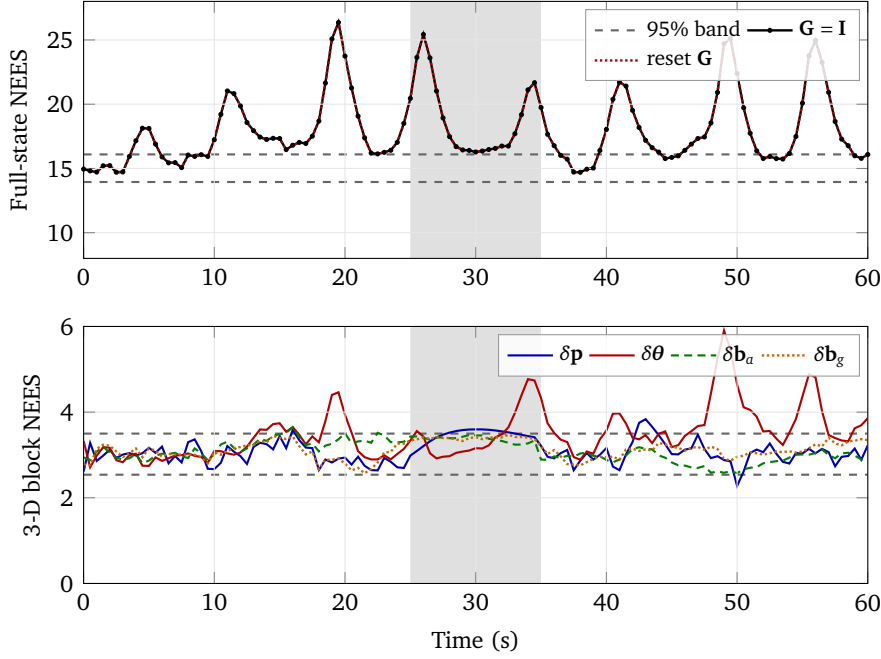
| $\rho$           | 0.20 | 0.25 | 0.30 | 0.35 | <b>0.40</b> | 0.45 | 0.50 | 0.60 |
|------------------|------|------|------|------|-------------|------|------|------|
| Accepted         | 100  | 98   | 94   | 84   | 78          | 73   | 67   | 54   |
| False accepts    | 3    | 2    | 0    | 0    | 0           | 0    | 0    | 0    |
| Rejected correct | 0    | 1    | 3    | 13   | 19          | 24   | 30   | 43   |

Table 7 by at most 0.026 at any epoch and in any block, and leaves every in/above/below count unchanged, so the omitted reset Jacobian is not the source of the inconsistency; with 10 Hz pose updates the injected attitude corrections are small, which is consistent with  $\mathbf{G} \approx \mathbf{I}$ . We have not isolated the cause; candidates are the zero-order-hold discretization of the synthetic IMU, which the process model does not account for, and the first-order treatment of the attitude–bias coupling.

## 5. Discussion and Limitations

**What the Factoring Buys.** Because the MCL core is written against one `MeasurementModel` interface, the 2D and 3D backends share the motion model, weighting, KLD resampling, estimate, and every test of those components; adding a backend means adding one likelihood function. This is the same benefit that Beluga’s shared core provides (Puga et al., 2024), and we report it as a property of the artifact rather than as a contribution. Because the cores are middleware-free, every equation of Section 3 is exercised by deterministic tests and by the closed-loop drivers of Section 4 without a ROS installation, which is also what makes the evaluation reproducible bit for bit from the repository.

**Cost Analysis.** The per-update cost of the MCL backends is linear in the particle count and in the number of scored beams or points (Equations (7) and (8)); KLD adaptation keeps the former at the 500-particle floor once the posterior is concentrated (Figure 2). The branch-and-bound matcher’s cost is data-dependent through pruning: on the synthetic room a full-map query took a median of 37.65 ms (min 8.77 ms, 95th percentile 106.07 ms) on the machine that produced the committed data, and it runs only when relocalization is requested, not every scan. Because the map-sized window grows with map area, so does the cost: on the 80 m map of Table 5 the median query takes 1526 ms, which is acceptable for an on-demand recovery but not for per-scan use; `bbs_max_linear_window` bounds it at the price of global coverage. Keeping the top eight modes weakens the pruning, and the local re-matches of the later scans are cheap, so in Table 6 a verified decision costs roughly 1.3 times



**Figure 5: Monte Carlo NEES of the ESKF on synthetic data** ( $N = 100$  runs of the Figure 4 scenario with initial errors and IMU biases drawn from  $\mathbf{P}_0$ ; aiding dropout shaded). Top: run-averaged NEES of the full 15-D error state for the shipped filter ( $\mathbf{G} = \mathbf{I}$ ) and with the reset Jacobian, which overlap; dashed lines bound the two-sided 95% interval. Bottom: 3-D blocks of the shipped filter against the 3-dof interval. The full-state NEES lies above the interval at most epochs and never below it; attitude is the most overconfident block, position and velocity exceed the interval mainly during and after the dropout, and both bias blocks stay inside it.

**Table 9: Ablation by clutter level.** The 100 BBS queries of Figure 3 split into thirds of the clutter range  $[0, 0.8]$  by the per-query `clutter_frac` column; success as in Table 4, acceptance at  $\rho = 0.40$ .

| Clutter probability | Queries | Success | Accepted | Median pos. err. (m) | Median score fraction |
|---------------------|---------|---------|----------|----------------------|-----------------------|
| $[0, 0.267)$        | 37      | 37      | 37       | 0.040                | 0.831                 |
| $[0.267, 0.533)$    | 30      | 30      | 30       | 0.032                | 0.624                 |
| $[0.533, 0.8]$      | 33      | 30      | 11       | 0.037                | 0.353                 |

the single-scan query on the 80 m map, plus the time to drive eight more scans. The ESKF step is a fixed  $15 \times 15$  covariance propagation per IMU sample.

**Limitations.** PRISM-Loc is a systems paper that integrates established components, and its scope is deliberately bounded. (i) *Planar state in the MCL backends.* Both particle-filter backends estimate  $(x, y, \theta)$ ; `ndt3d` evaluates a full 3D Gaussian-voxel map from that planar pose at a fixed base height, which suits wheeled robots on locally flat floors but does not track roll, pitch, or vertical motion. Full  $SE(3)$  state exists only in `fusion3d`; existing particle-filter packages already estimate a full 3D pose against NDT or point-cloud maps (Ekumen, Inc., 2022; Puga et al., 2024; Watanabe, 2016). (ii) *2D-only relocalization with a heuristic verification.* Algorithm 2 operates on a 2D occupancy pyramid; `ndt3d` and `fusion3d` rely on an operator-supplied or GNSS-derived initial pose and local tracking. The search window is sized from the map and restricted to non-occupied cells. On maps with repeated structure the score fraction of Equation (11) alone does not separate look-alike places: a single scan accepted a wrong pose on 6 of 120 (40 m) and 15 of 120 (80 m) synthetic queries, and no single threshold removes these on the two maps together without rejecting more than half of the correct fixes. Multi-scan verification (Section 3.5) reduces them to 0 and 1, but it is a heuristic:

its evidence score (named a posterior in the code) treats correlated scans as independent evidence behind a tuned temperature and is not a calibrated probability, its parameters were tuned on one synthetic generator (on disjoint seeds), the one residual false accept held 0.969 of the evidence score, it can only choose among the  $K$  modes of the first scan, it needs the robot to move, and the top-8 search is slower than the single-best one. It was evaluated only on synthetic worlds with simulated odometry. Relocalization runs only at startup or on a service call; unlike reliability-aware MCL (Akai, 2023), no loss detector triggers it. Beyond the compute cap (100 m per axis by default) the search is no longer global, which the node reports at startup. (iii) *No time-offset or extrinsic estimation*. The ESKF treats the LiDAR/GNSS-to-IMU time offset and the LiDAR-IMU extrinsic as known inputs. (iv) *Loosely-coupled LiDAR update and an inconsistent covariance*. In `fusion3d`, PCL NDT produces a 6-DoF pose measurement that the ESKF fuses as a loosely-coupled update, which keeps the filter core independent of the registration engine but forgoes the accuracy of tightly-coupled formulations such as FAST-LIO (Xu and Zhang, 2021). The filter is also not NEES-consistent on our synthetic scenario (Table 7): it is overconfident, most of all in attitude and, while aiding is absent, in position and velocity, whereas its bias blocks are consistent; the optional reset Jacobian of Solà (2017) does not change this. We therefore regard `fusion3d` as experimental: consumers should treat the published orientation covariance, and the position covariance during aiding outages, with caution until the cause is found. (v) *Single static map*. All backends assume one prior map of a static world, loaded once, with no online map update or change detection. (vi) *No external baseline and no real data*. The evaluation of Section 4 is deterministic unit and integration testing plus a seeded synthetic study whose worlds and sensor-noise models match the filters’ assumptions by construction. Every comparison in it is internal, between configurations of PRISM-Loc: we have not compared the trackers with `nav2_amcl`, Beluga (Puga et al., 2024), or MH-AMCL (García et al., 2023), the relocalizer with `hdl_global_localization` (Koide, 2021), the recovery tooling of `lidar_localization_ros2` (Sasaki, 2026), CBGL (Filotheou, 2024), or the method of Zhang et al. (2026a), or the verifier with the multiple-hypothesis tracker of Gao et al. (2019). The study therefore establishes that each core behaves as specified in closed loop; it does not establish accuracy on a physical robot, competitiveness with these systems, or that the false-accept reduction transfers to real maps and real odometry. We have not measured trajectory error on physical or public datasets. The planned evaluation runs the three backends and at least `nav2_amcl` and one external global-localization method on public LiDAR sequences, including one with repeated structure, and reports absolute and relative trajectory error (ATE/RPE) following the TUM RGB-D conventions (Sturm et al., 2012) on the per-backend map→odom estimates, together with relocalization success and false-accept rates. Until then we make no accuracy claim.

**Future Work.** Extending the MCL core to a 6-DoF particle state or coupling it to an external attitude source, re-admitting new candidates during verification and a principled (correlation-aware) evidence model for the 2D relocalizer, a loss detector that triggers it, a 3D branch-and-bound front end such as 3D-BBS (Aoki et al., 2024) or a place-recognition front end for the 3D backends, a diagnosis of the ESKF’s attitude overconfidence, joint time-offset and extrinsic estimation in the ESKF, and the external-baseline evaluation above are the next steps on the roadmap.

## 6. Conclusion

We presented PRISM-Loc, a LiDAR localization stack whose contribution is a specific integration of established components rather than a new algorithm: three known estimator families, likelihood-field MCL, NDT-MCL (Saarinen et al., 2013), and an error-state Kalman filter fusing LiDAR, IMU, and RTK-GNSS, sit behind a single ROS 2 topic and TF contract, so that a 2D LiDAR, a 3D LiDAR, or a sensor-fused configuration is a deployment choice rather than a switch to a stack with a different

interface. Each estimator is a pure C++ core with no dependence on the middleware runtime, the two MCL backends share one predict-weight-resample core that differs only in its measurement model, as in existing MCL libraries (Puga et al., 2024), and `laser2d` relocalizes deterministically by branch-and-bound (Hess et al., 2016) over a map-sized window of non-occupied cells and verifies the top candidates over several scans before committing, a simple instance of established multi-hypothesis verification that can report a place as ambiguous. Seeded synthetic experiments show that the assembled pipelines behave as intended in closed loop, and they also expose two open problems that we report rather than hide: on large maps with repeated structure a single scan admits wrong poses, which multi-scan verification makes rare (1 of 240 synthetic queries) but does not rule out, and the ESKF covariance is overconfident, mainly in attitude. An evaluation on real and public data against external baselines remains future work. PRISM-Loc is released under the Apache-2.0 license at [https://github.com/kjungmo/prism\\_loc](https://github.com/kjungmo/prism_loc).

## References

- N. Adurthi. Scan matching-based particle filter for LIDAR-only localization. *Sensors*, 23(8):4010, 2023. doi: 10.3390/s23084010.
- N. Akai. Reliable Monte Carlo localization for mobile robots. *Journal of Field Robotics*, 40(3):595–613, 2023. doi: 10.1002/rob.22149.
- K. Aoki, K. Koide, S. Oishi, M. Yokozuka, A. Banno, and J. Meguro. 3D-BBS: Global localization for 3D point cloud scan matching using branch-and-bound algorithm. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1796–1802. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10610810.
- Autoware Foundation. `autoware_pose_estimator_arbiter`. Autoware Universe documentation, [https://autowarefoundation.github.io/autoware\\_universe/main/localization/autoware\\_pose\\_estimator\\_arbiter/](https://autowarefoundation.github.io/autoware_universe/main/localization/autoware_pose_estimator_arbiter/), 2024. Accessed 2026-09-30.
- Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. John Wiley & Sons, New York, 2001. doi: 10.1002/0471221279.
- P. Biber and W. Straßer. The normal distributions transform: A new approach to laser scan matching. In *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2743–2748, Las Vegas, NV, USA, Oct. 2003. IEEE. doi: 10.1109/IROS.2003.1249285.
- J. L. Blanco-Claraco. A flexible framework for accurate LiDAR odometry, map manipulation, and localization. *The International Journal of Robotics Research*, 44(9):1553–1599, 2025. doi: 10.1177/02783649251316881.
- F. Dellaert, D. Fox, W. Burgard, and S. Thrun. Monte Carlo localization for mobile robots. In *Proceedings of the 1999 IEEE International Conference on Robotics and Automation (ICRA)*, volume 2, pages 1322–1328, Detroit, MI, USA, May 1999. IEEE. doi: 10.1109/ROBOT.1999.772544.
- Ekumen, Inc. `Beluga`. GitHub repository Ekumen-OS/beluga, <https://github.com/Ekumen-OS/beluga>, 2022. Generic C++17 MCL library with ROS AMCL nodes that provide interface parity with `nav2_amcl` (and `amcl`); includes likelihood-field, beam, and NDT sensor models and 2D and 3D NDT AMCL nodes for ROS 2; accessed 2026-09-30.

- P. F. Felzenszwalb and D. P. Huttenlocher. Distance transforms of sampled functions. *Theory of Computing*, 8(19):415–428, 2012. doi: 10.4086/toc.2012.v008a019.
- A. Filotheou. CBGL: Fast Monte Carlo passive global localisation of 2D LIDAR sensor. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3268–3275. IEEE, 2024. doi: 10.1109/IROS58592.2024.10802235.
- D. Fox. Adapting the sample size in particle filters through KLD-sampling. *The International Journal of Robotics Research*, 22(12):985–1003, 2003. doi: 10.1177/0278364903022012001.
- D. Fox, W. Burgard, F. Dellaert, and S. Thrun. Monte Carlo localization: Efficient position estimation for mobile robots. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI/IAAI)*, pages 343–349, Orlando, FL, USA, 1999. AAAI Press / The MIT Press.
- H. Gao, X. Zhang, J. Yuan, J. Song, and Y. Fang. A novel global localization approach based on structural unit encoding and multiple hypothesis tracking. *IEEE Transactions on Instrumentation and Measurement*, 68(11):4427–4442, Nov. 2019. doi: 10.1109/TIM.2018.2890455.
- A. García, F. Martín, J. M. Guerrero, F. J. Rodríguez, and V. Matellán. Portable multi-hypothesis Monte Carlo localization for mobile robots. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1933–1939. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10160957.
- W. Hess, D. Kohler, H. Rapp, and D. Andor. Real-time loop closure in 2D LIDAR SLAM. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, Stockholm, Sweden, 2016. IEEE. doi: 10.1109/ICRA.2016.7487258.
- K. Koide. hdl\_global\_localization. ROS package, GitHub repository, [https://github.com/koide3/hdl\\_global\\_localization](https://github.com/koide3/hdl_global_localization), 2021. Global localization service with 2D grid-map branch-and-bound, FPFH+RANSAC, and FPFH+TEASER++ engines; accessed 2026-09-30.
- K. Koide, J. Miura, and E. Menegatti. A portable three-dimensional LiDAR-based system for long-term and wide-area people behavior measurement. *International Journal of Advanced Robotic Systems*, 16(2), Mar. 2019. ISSN 1729-8814. doi: 10.1177/1729881419841532.
- Locus Robotics. fuse. GitHub repository, <https://github.com/locusrobotics/fuse>, 2018. A ROS framework for sensor fusion using nonlinear least squares optimization; accessed 2026-09-30.
- S. Macenski, F. Martín, R. White, and J. G. Clavero. The Marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2718–2725. IEEE, 2020. doi: 10.1109/IROS45743.2020.9341207.
- S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson. From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2. *Robotics and Autonomous Systems*, 168:104493, 2023. doi: 10.1016/j.robot.2023.104493.
- M. Magnusson, A. Lilienthal, and T. Duckett. Scan registration for autonomous mining vehicles using 3D-NDT. *Journal of Field Robotics*, 24(10):803–827, 2007. ISSN 1556-4959. doi: 10.1002/rob.20204.
- F. Martín Rico, J. M. Guerrero Hernández, R. Pérez Rodríguez, J. D. Peña Narváez, and A. García Gómez-Jacinto. Open source robot localization for non-planar environments. arXiv preprint arXiv:2309.12744, 2023.

- T. Moore and D. Stouch. A generalized extended Kalman filter implementation for the robot operating system. In *Intelligent Autonomous Systems 13: Proceedings of the 13th International Conference (IAS-13)*, volume 302 of *Advances in Intelligent Systems and Computing*, pages 335–348, Cham, 2016. Springer. ISBN 978-3-319-08338-4. doi: 10.1007/978-3-319-08338-4\_25.
- E. B. Olson. Real-time correlative scan matching. In *2009 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4387–4393, Kobe, Japan, 2009. IEEE. doi: 10.1109/ROBOT.2009.5152375.
- E. Pedrosa. *iris\_lama*. GitHub repository, [https://github.com/iris-ua/iris\\_lama](https://github.com/iris-ua/iris_lama), 2019. C++ localization and mapping library whose only dependency is Eigen; accessed 2026-09-30.
- G. Puga, N. Espinosa, M. Hidalgo, O. García, and I. Paunovic. Beluga: A modern Monte Carlo localization package for ROS and ROS 2. In *European Robotics Forum 2024*, Springer Proceedings in Advanced Robotics, pages 126–130, Cham, 2024. Springer Nature Switzerland. doi: 10.1007/978-3-031-76424-0\_23.
- J. Saarinen, H. Andreasson, T. Stoyanov, and A. J. Lilienthal. Normal distributions transform Monte-Carlo localization (NDT-MCL). In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 382–389, Tokyo, Japan, Nov. 2013. IEEE. doi: 10.1109/IROS.2013.6696380.
- R. Sasaki. *lidar\_localization\_ros2: Global localization*. Documentation in the *lidar\_localization\_ros2* GitHub repository, [https://github.com/rsasaki0109/lidar\\_localization\\_ros2/blob/main/docs/global\\_localization.md](https://github.com/rsasaki0109/lidar_localization_ros2/blob/main/docs/global_localization.md), 2026. The repository dates from 2020; this document was added in 2026. Accessed 2026-09-30.
- J. Solà. Quaternion kinematics for the error-state Kalman filter, Nov. 2017. URL <https://arxiv.org/abs/1711.02508>. arXiv:1711.02508 [cs.RO], tutorial/technical report.
- J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 573–580, Vilamoura, Portugal, Oct. 2012. IEEE. doi: 10.1109/IROS.2012.6385773.
- S. Thrun, D. Fox, W. Burgard, and F. Dellaert. Robust Monte Carlo localization for mobile robots. *Artificial Intelligence*, 128(1-2):99–141, May 2001. ISSN 0004-3702. doi: 10.1016/S0004-3702(01)00069-8.
- S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA, 2005. ISBN 978-0-262-20162-9.
- G. Wan, X. Yang, R. Cai, H. Li, Y. Zhou, H. Wang, and S. Song. Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4670–4677, Brisbane, QLD, Australia, May 2018. IEEE. doi: 10.1109/ICRA.2018.8461224.
- A. Watanabe. *mcl\_3dl*. ROS package, GitHub repository, [https://github.com/at-wat/mcl\\_3dl](https://github.com/at-wat/mcl_3dl), 2016. 3-D/6-DOF point-cloud Monte Carlo localization; accessed 2026-09-30.
- E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- W. Xu and F. Zhang. FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter. *IEEE Robotics and Automation Letters*, 6(2):3317–3324, Apr. 2021. doi: 10.1109/LRA.2021.3064227.

- M. Zhang, L. Ma, Y. Wu, K. Shen, D. Huang, and H. Leung. Tackling the kidnapped robot problem via sparse feasible hypothesis sampling and reliable batched multistage inference. *IEEE Transactions on Instrumentation and Measurement*, 75:7504614, 2026a. doi: 10.1109/TIM.2026.3694741.
- Y. Zhang, Y. Guo, J. Zheng, Z. Fan, X. Jia, and J. Liu. Move first, commit later: Selective LiDAR-to-BIM global initialization via sequential consensus with symmetry-aware abstention. arXiv preprint arXiv:2607.17103, July 2026b. Concurrent work.
- X. Zou, J. Li, W. Wu, F. Liang, B. Yang, and Z. Dong. Reliable-loc: Robust sequential LiDAR global localization in large-scale street scenes based on verifiable cues. arXiv preprint arXiv:2411.07815, 2024.

## A. Reproduction

Every number and figure of Section 4 regenerates from the repository alone. The drivers and the core test suites need only a C++17 compiler and the Eigen 3 headers (Debian/Ubuntu packages `g++`, `libeigen3-dev`); the unit tests additionally use GoogleTest (`libgtest-dev`). ROS 2 Humble is required only for the two node packages, which continuous integration builds in a `ros:humble` container (`.github/workflows/ci.yml`). From the repository root:

```
CORE="-I prism_loc_core/include -I prism_loc_core/test -I /usr/include/eigen3"
g++ -O2 -std=c++17 $CORE prism_loc_core/src/*.cpp \
  experiments/mcl_tracking.cpp -o /tmp/exp_mcl && /tmp/exp_mcl
g++ -O2 -std=c++17 $CORE prism_loc_core/src/*.cpp \
  experiments/bbs_relocalization.cpp -o /tmp/exp_bbs && /tmp/exp_bbs
g++ -O2 -std=c++17 -I prism_loc_fusion/include -I /usr/include/eigen3 \
  prism_loc_fusion/src/*.cpp experiments/eskf_fusion.cpp -o /tmp/exp_eskf
/tmp/exp_eskf > docs/paper/data/eskf_track.csv
g++ -O2 -std=c++17 $CORE -I experiments prism_loc_core/src/*.cpp \
  experiments/legacy/bbs_v01.cpp experiments/bbs_largemap.cpp \
  -o /tmp/exp_bbs_large && /tmp/exp_bbs_large # about 7 min
g++ -O2 -std=c++17 -I prism_loc_fusion/include -I /usr/include/eigen3 \
  prism_loc_fusion/src/*.cpp experiments/eskf_nees.cpp -o /tmp/exp_nees
/tmp/exp_nees
g++ -O2 -std=c++17 $CORE prism_loc_core/src/*.cpp \
  experiments/bbs_verify.cpp -pthread -o /tmp/exp_bbs_verify
/tmp/exp_bbs_verify calib 10 # parameter-selection log
python3 experiments/select_verify_params.py --check
/tmp/exp_bbs_verify study 3 # verification study
python3 docs/paper/make_study_tables.py # generated study tables
python3 docs/paper/check_numbers.py # recompute every quoted statistic
```

All randomness derives from fixed compile-time seeds (Table 10), so every CSV column is bit-for-bit reproducible except the wall-clock `time_ms` latency columns of the two BBS experiments. We re-ran all three drivers from the sources of this commit while preparing this version: `mcl_tracking.csv`, `mcl_particles.csv`, and `eskf_track.csv` were byte-identical to the committed files, and `bbs_relocalization.csv` matched in every column except `time_ms`. On that re-run machine (AMD Ryzen 5 5500GT, single thread) the median BBS query latency was 10.26 ms against the 37.65 ms of the committed CSV, which illustrates why latency is reported as machine-dependent; the re-run CSV is committed under `docs/paper/data/rerun_2026-09-28/`. After the matcher gained its symmetric window and feasibility rule we re-ran the room study once more against the current code (`bbs_relocalization_fixbbs.csv` in the same directory): again every column except `time_ms` is identical to the committed CSV, with a median latency of 9.90 ms on the same re-run machine. The guard script `docs/paper/check_numbers.py` recomputes each headline statistic from the committed CSVs and fails if a number quoted in the  $\text{\LaTeX}$  sources, including this package, no longer matches.

## B. Parameters

Table 11 lists the defaults used throughout the paper; each is a declared ROS 2 parameter documented one-to-one in `PARAMS.md`, with deployed overrides in `prism_loc/params/laser2d.yaml`, `prism_loc/params/ndt3d.yaml`, and `prism_loc_fusion_ros/params/fusion3d.yaml`.

**Table 10: Experiment drivers, fixed seeds, and committed artifacts.** Drivers live under `experiments/`; CSVs under `docs/paper/data/` (copied to `data/` in this package).

| Driver                              | Seed(s)        | Figures / tables            | CSV(s)                                                               |
|-------------------------------------|----------------|-----------------------------|----------------------------------------------------------------------|
| <code>mcl_tracking.cpp</code>       | 101, 202, 303  | Figure 2, Table 3           | <code>mcl_tracking.csv</code> ,<br><code>mcl_particles.csv</code>    |
| <code>bbs_relocalization.cpp</code> | 20240607       | Figure 3, Tables 4, 8 and 9 | <code>bbs_relocalization.csv</code>                                  |
| <code>eskf_fusion.cpp</code>        | 20240517       | Figure 4, Table 4           | <code>eskf_track.csv</code>                                          |
| <code>bbs_largemap.cpp</code>       | 20260928       | Table 5                     | <code>bbs_largemap.csv</code>                                        |
| <code>bbs_verify.cpp</code>         | 20260928       | Table 6                     | <code>bbs_verify.csv</code>                                          |
| parameter selection                 | 20261105       | Section 3.5                 | <code>bbs_verify_calib.csv</code> ,<br><code>.._selection.csv</code> |
| <code>eskf_nees.cpp</code>          | 20260929 + $k$ | Table 7, Figure 5           | <code>eskf_nees.csv</code>                                           |

## C. Code Map

Table 12 maps each equation and algorithm to the source file that implements it, and Table 13 collects the notation.

**Table 11: Default parameters** (from the shipped YAML files; symbols as in Section 3).

| Parameter                             | Symbol                                                   | Default                             | Used in                  |
|---------------------------------------|----------------------------------------------------------|-------------------------------------|--------------------------|
| min_particles / max_particles         | $N_{\min} / N_{\max}$                                    | 500 / 2000                          | Equation (5)             |
| resample_threshold                    | $\tau$                                                   | 0.5                                 | $N_{\text{eff}}$ trigger |
| kld_err / kld_z                       | $\epsilon_{\text{kld}} / z_{1-\delta}$                   | 0.05 / 2.33                         | Equation (5)             |
| kld_bin_xy / kld_bin_yaw              | –                                                        | 0.5 m / 0.17 rad                    | Equation (5)             |
| update_min_d / update_min_a           | $d_{\min} / a_{\min}$                                    | 0.2 m / 0.2 rad                     | Algorithm 1              |
| alpha1-alpha4                         | $\alpha_{1..4}$                                          | 0.2                                 | Equation (3)             |
| max_beams                             | –                                                        | 60                                  | Equation (7)             |
| z_hit / z_rand / sigma_hit            | $z_{\text{hit}} / z_{\text{rand}} / \sigma_{\text{hit}}$ | 0.5 / 0.5 / 0.2 m                   | Equations (7) and (9)    |
| likelihood_max_dist                   | –                                                        | 2.0 m                               | Equation (7)             |
| ndt_resolution / voxel_min_points     | –                                                        | 1.0 m / 5                           | Equation (8)             |
| max_points / base_height              | –                                                        | 500 / 0.0 m                         | Equation (8)             |
| bbs_global_window                     | –                                                        | true                                | Section 3.4              |
| bbs_max_linear_window                 | $W_{\max}$                                               | 50.0 m                              | Section 3.4              |
| bbs_linear_window (if not global)     | $W$                                                      | 10.0 m                              | Section 3.4              |
| bbs_angular_window / bbs_angular_step | –                                                        | $\pi / 0.0175$ rad                  | Algorithm 2              |
| bbs_max_depth / bbs_max_beams         | $D / -$                                                  | 6 / 120                             | Equations (9) and (10)   |
| bbs_min_score_fraction                | $\rho$                                                   | 0.4                                 | Equation (11)            |
| bbs_verify_top_k                      | $K$                                                      | 8                                   | Section 3.5              |
| bbs_verify_scans                      | $M$                                                      | 9                                   | Section 3.5              |
| bbs_verify_evidence_gain              | $\lambda$                                                | 40                                  | Equation (12)            |
| bbs_verify_min_posterior              | $\tau$                                                   | 0.9                                 | Equation (12)            |
| bbs_verify_nms_xy / _yaw              | –                                                        | 1.0 m / 0.35 rad                    | Section 3.5              |
| bbs_verify_track_*_window             | –                                                        | 0.3 m / 0.1 rad                     | Section 3.5              |
| sigma_acc / sigma_gyro                | $\sigma_a / \sigma_g$                                    | 0.01 / 0.001                        | Equation (13)            |
| sigma_acc_bias / sigma_gyro_bias      | $\sigma_{b_a} / \sigma_{b_g}$                            | $10^{-4} / 10^{-5}$                 | Equation (13)            |
| reset_jacobian                        | $\mathbf{G}$                                             | false ( $\mathbf{G} = \mathbf{I}$ ) | Section 3.6              |
| pose_pos_std / pose_rot_std           | $\sigma_{\text{pos}} / \sigma_{\text{rot}}$              | 0.1 m / 0.05 rad                    | Equation (14)            |
| ndt_max_fitness                       | –                                                        | 2.0                                 | NDT gate                 |
| gnss_min_status / gnss_max_pos_cov    | $s_{\min} / c_{\max}$                                    | 0 / 25.0 m <sup>2</sup>             | GNSS gate                |
| transform_tolerance / seed            | –                                                        | 0.1 s / 42                          | Equation (2) / RNG       |

**Table 12: Where each equation lives.** Paths are relative to the repository root; `core/` abbreviates `prism_loc_core/src/`, `fusion/` abbreviates `prism_loc_fusion/src/`, and `node/` and `fnode/` abbreviate `prism_loc/src/localization_node.cpp` and `prism_loc_fusion_ros/src/fusion_localization_node.cpp`.

| Equation / algorithm                  | Source                                                                                                                                                                                                                                                                                        |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Equation (2) (map→odom)               | <code>node/</code> , <code>fnode/</code> ; tests <code>prism_loc/test/test_tf_math.cpp</code> , <code>prism_loc_fusion_ros/test/test_tf_compose.cpp</code>                                                                                                                                    |
| Equation (3) (motion model)           | <code>core/motion_model.cpp</code>                                                                                                                                                                                                                                                            |
| Equations (4) to (6) (filter)         | <code>core/particle_filter.cpp</code>                                                                                                                                                                                                                                                         |
| Algorithm 1 (update order, gates)     | <code>node/</code> ( <code>runUpdate</code> , <code>onScan</code> )                                                                                                                                                                                                                           |
| Equation (7) (likelihood field)       | <code>core/measurement_model.cpp</code> , <code>core/occupancy_grid.cpp</code>                                                                                                                                                                                                                |
| Equation (8) (NDT score)              | <code>core/measurement_model.cpp</code> , <code>core/ndt_map.cpp</code>                                                                                                                                                                                                                       |
| Equations (9) to (11), Algorithm 2    | <code>core/bbs.cpp</code> ; re-seeding in <code>node/</code> ( <code>runRelocalization</code> )                                                                                                                                                                                               |
| Equations (13) and (14) (ESKF)        | <code>fusion/eskf.cpp</code> , <code>fusion/so3.cpp</code> , <code>fusion/geodetic.cpp</code>                                                                                                                                                                                                 |
| Equation (15), GNSS and NDT gates     | <code>fnode/</code> ( <code>attitudeFromAccel</code> , <code>onGnss</code> , <code>onPoints</code> )                                                                                                                                                                                          |
| Corollary 1 check (exhaustive search) | <code>core/bbs.cpp</code> ( <code>matchExhaustive</code> ); test <code>prism_loc_core/test/test_bbs.cpp</code>                                                                                                                                                                                |
| Equation (12), Section 3.5            | <code>core/relocalization.cpp</code> ( <code>RelocalizationVerifier</code> ), <code>core/bbs.cpp</code> ( <code>matchGlobalTopK</code> , <code>matchLocal</code> ); <code>node</code> <code>runRelocalization</code> ; test <code>test_relocalization.cpp</code>                              |
| Synthetic drivers                     | <code>experiments/mcl_tracking.cpp</code> , <code>bbs_relocalization.cpp</code> , <code>eskf_fusion.cpp</code> , <code>bbs_largemap.cpp</code> (with <code>legacy/bbs_v01.cpp</code> ), <code>eskf_nees.cpp</code> , <code>bbs_verify.cpp</code> (with <code>select_verify_params.py</code> ) |

**Table 13: Notation.**

| Symbol                                      | Meaning                           | Symbol                                 | Meaning                                           |
|---------------------------------------------|-----------------------------------|----------------------------------------|---------------------------------------------------|
| $\mathcal{B}$                               | set of backends                   | $\mathbf{T}_{a,b}$                     | pose of frame $b$ in frame $a$                    |
| $\mathbf{x} = (x, y, \theta)$               | planar particle pose              | $w^{[i]}$                              | particle weight                                   |
| $\ell(\mathbf{x})$                          | measurement log-likelihood        | $N_{\text{eff}}$                       | effective sample size                             |
| $M_k$                                       | KLD sample bound, $k$ bins        | $d(\cdot)$                             | distance to nearest obstacle                      |
| $\boldsymbol{\mu}_v, \boldsymbol{\Sigma}_v$ | NDT voxel mean, covariance        | $P_h$                                  | BBS pyramid level $h$                             |
| $S_h(k, \mathbf{o})$                        | BBS node score                    | $\mathcal{E}$                          | subsampled scan endpoints                         |
| $\rho$                                      | BBS acceptance fraction           | $D$                                    | BBS pyramid depth                                 |
| $O, L_x, L_y$                               | BBS offset set, half-windows      | $\mathcal{F}$                          | feasible BBS candidates                           |
| $W_{\text{max}}$                            | BBS half-window cap               | $\epsilon$                             | NEES $\mathbf{e}^\top \mathbf{P}^{-1} \mathbf{e}$ |
| $\mathbf{p}, \mathbf{v}, \mathbf{q}$        | ESKF position, velocity, attitude | $\mathbf{b}_a, \mathbf{b}_g$           | IMU biases                                        |
| $\delta \mathbf{x} \in \mathbb{R}^{15}$     | ESKF error state                  | $\mathbf{P}$                           | error covariance                                  |
| $\mathbf{F}, \mathbf{Q}$                    | transition, process noise         | $\mathbf{H}, \mathbf{R}_z, \mathbf{K}$ | update Jacobian, noise, gain                      |
| $\mathbf{G}$                                | ESKF reset Jacobian               |                                        |                                                   |